



POZNAN UNIVERSITY OF TECHNOLOGY

FACULTY OF COMPUTING AND TELECOMMUNICATION
Institute of Computing Science

Master's thesis

DEVELOPMENT OF GENETIC REPRESENTATIONS FOR OPTIMIZATION OF 3D STRUCTURES USING GRAPH AUTOENCODERS

Adam Klejda, 131778

Supervisor
dr hab. inż. Maciej Komosiński

POZNAŃ 2021

I would like to express my gratitude towards my supervisor dr hab. inż. Maciej Komosiński for his guidance during the development of this thesis.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Scope of the thesis	2
1.3	The structure of the thesis	2
2	Background theory	3
2.1	Optimization	3
2.1.1	Evolutionary algorithms	3
2.1.2	Population	4
2.1.3	Mutation and cross over	5
2.1.4	Selection	5
2.1.5	Fitness function	5
2.2	Framsticks	5
2.2.1	Agents	6
2.3	Genetic representations	6
2.3.1	Genetic representations in Framsticks	6
2.3.2	Genotypes and phenotypes	8
2.3.3	Impact of encoding	9
2.4	Deep Learning	9
2.4.1	Autoencoders	10
2.4.2	Graph neural networks	12
2.5	Latent representation	14
3	The proposed approach	16
3.1	Architecture	16
3.1.1	Environment	17
3.1.2	Graph data-set	17
3.1.3	Autoencoder	18
	Encoder GAE	18
	Encoder VGAE	19
	Decoder of A'	20
	Decoder of X'	20
	Reconstruction Loss	21
3.1.4	Locality loss manager	21
3.1.5	Post processing	22
3.2	Data	23
3.3	Conducted experiments	24
3.3.1	Architecture development	25

3.3.2	Locality loss experiments	25
3.3.3	Evolutionary experiments	26
3.4	Description of landscape analysis methods	27
3.4.1	Mutation impact	27
3.4.2	Convexity of latent space	27
4	Experimental results	28
4.1	Development of autoencoders	28
4.1.1	Latent space properties	31
4.2	Development locality loss	34
4.2.1	Locality based on the number of parts	35
4.2.2	Locality based on similarity measure	36
4.2.3	Locality based on fitness	36
4.2.4	Summary of locality losses	38
4.3	Vertpos data set	41
4.4	Evolutionary experiments	45
5	Conclusions	48
5.1	Summary	48
5.2	Key points	48
5.3	Future work	49
	Bibliography	50

Chapter 1

Introduction

During solving any optimization problem, it is important to correctly define the possible solutions and the rules that these solutions must follow. As the solution is defined and expressed in some kind of representation, an optimization process can be performed. Optimization can be performed by some expert directly modelling solutions, or by some automated algorithm. In both cases, the optimization process relies on modifying some starting solution and testing its new properties. An example of this can be building a robot. Expert knowing available resources and the purpose of a robot can create a project based on his knowledge. Projects created that way can fulfil the base goals, but it can also be far from the best possible solution. To check if there is a better solution, changes would have to be proposed and implemented, then new solution should be compared with the old one, using some metric. While trying to optimize the problem some changes can be more intuitive than the others. Let us say that the robot has to move some objects using his artificial arms. Adding another arm might increase the productivity of a robot, but what if giving a robot better manoeuvrability at the cost of reducing number of arms would increase it even higher. The change from one approach to the other can be very different, based on what representation of the solution is used. In some representations, the change can be bigger than the others. This can cause problems, especially in automated approaches, as these focus on the move through space of solutions by analyzing the surroundings, which have similar solutions. But what if the solution space was defined in a way, that the translation from one individual to the other was different, and the translation from multi-armed robot to the robot with a lot of manoeuvrability would be more smooth. In that case, the optimization algorithm would have it much easier to find that translation as it would be in its neighbourhood.

This thesis proposes an architecture that learns the problem based on given examples of solutions. Then using learned representation it moves through new created space and generates new solutions.

1.1 Motivation

The main goal of this thesis is to design and implement an architecture that will be able to learn given representation, and enable evolutionary algorithms to reach new solutions. To achieve that, mentioned architecture has to be able to learn properties of original representation and allow for an evolutionary walk in different dimensions. The proposed solution uses graph-based representation, because of a big spectrum of applications of mention representation. Another reason for using graph representation is to be able to preserve relations taking place in represented objects or problems. To create a representation of a problem graph-based autoencoder architecture is proposed, as

autoencoder architectures demonstrated great capabilities in learning subspace of reconstructed data.

1.2 Scope of the thesis

The scope of this thesis consists of developing autoencoder based architecture which will be able to learn on graph-based representation. The thesis considers two approaches to auto encoder architecture, traditional *GAE* and variational *VGAE*. Furthermore, the thesis presents and test three different locality loss type whose job is to smooth the landscape. Models are tested towards reconstruction capabilities, the impact of mutation made on learned latent space, and the convexity of latent space. The thesis includes also evolutionary tests made on learned models.

1.3 The structure of the thesis

The thesis consists of 5 chapters:

- Chapter 2 focuses on background literature, describing topics related to the developed architecture.
- Chapter 3 presents constructed architecture, describes used data, and explains what experiments were performed.
- Chapter 4 contains experimental results and explains them.
- Chapter 5 presents summary and conclusions as well as suggestions for future work.

Chapter 2

Background theory

This chapter explains the core concepts corresponding to issues and solutions presented in the thesis.

2.1 Optimization

One of the most important aspects of solving engineering problems is optimizing the solution. Based on the type and specification of the problem, there can be one or multiple best solutions. Often it is very hard to determine if the solution is the best possible and the only way to deny that is to find a better one. The simplest way of solving problems would be just to generate every possible solution, then test it and choose the best one. Unfortunately, an exhausting search is often too demanding from a computational perspective. Searching solutions for complicated problems using the brute force approach can take too much time and resources. That is why there are multiple approaches specializing in saving resources required for search at the expense of the quality of the solution. For optimization problems, gradient-based solutions can be used, but they might struggle if landscape has multiple local optima. Based on the problem, the algorithm might get stuck in the local optimum and won't be able to search for other, possibly better outcomes. To solve the optimization problems many approaches have arisen [3]:

- Unconstrained Optimization,
- 1-D Optimization Algorithms,
- Linear Programming,
- Constrained Optimization,
- Multi objective Optimization.

2.1.1 Evolutionary algorithms

By observing the biological world and trying to adapt occurring in its interactions, the idea for *Evolutionary Algorithms (EAs)* was born. *EAs* [45] are population-based algorithms, in population there is a group of individuals consisting of genetic representation. Each individual can be scored using a fitness function that is adjusted to the problem being solved. There are different branches of *EAs* [24] such as:

1. Genetic Algorithms [20]
2. Evolutionary Programming [13]

3. Genetic Programming [31]

4. Evolutionary Strategies [6]

Even though various variants of EA can differ from each other, their structure is basically the same [11]. An example of such structure is visible on Algorithm 1. The algorithm stores population P for iteration i . Based on the problem each individual represents a solution that might be presented in different representations. In each iteration, the population is evaluated. Based on evaluation individuals are selected and multiplied [24]. The selection mechanism might be different depending on the problem and algorithm used. On selected new population mutation is performed. These steps are repeated until a termination condition is fulfilled.

```

 $i \leftarrow 0$ ;
initialize  $P(i)$ ;
evaluate  $P(i)$ ;
while not termination-condition do
     $i \leftarrow i + 1$ ;
    select  $P(i - 1)$ ;
    mutate  $P(i)$ ;
    evaluate  $P(i)$ ;
end

```

Algorithm 1: Typical structure for EA

The main use of such meta-heuristics is to solve problems that are hard to solve using the exhausting approach, and there is no simple meta-heuristic solution that solves it. Possible applications for EA are optimization of mathematical functions [24], robotic design [32], multiple-criteria optimization [10] and biological simulation of evolution [24] as well as other biological relationship [35].

Great example of a problem and solution using EA is *Traveling Salesman Problem (TSP)* [12]. In this problem as the population can be seen as different possible permutations. Fitness can be defined as the cost of the travel for selected permutation, and mutation as swapping or moving points in permutation.

To be in control of EA there are multiple parameters (the number of them might vary depending on the algorithm). The most common parameters are [24]:

- population size,
- probability of mutation and crossing-over,
- selection mechanism and its parameters,
- stopping criterion.

2.1.2 Population

The population consists of individuals that can represent various solutions for a problem. The size of the population has a big impact on the functioning of EA, the bigger population the more solutions we can check in each step of the algorithm. With a bigger population, bigger space is searched but also computational complexity rises. There are solutions that allow population size to be changed while the algorithm is running [11] or to be constant.

2.1.3 Mutation and cross over

Mutation and *cross-over* are the main operations responsible for searching through new solution. *Mutation* inherits information from his ancestors and makes some changes based on available mutation operators. *Cross-over* inherits information from two or more ancestors and mix them together by swapping random parts of genotypes [24]. By adjusting mutation and cross over operators it is possible to narrow the space of searched solutions. Some *mutations* can cause unnecessary changes which will not impact final fitness. An example of such behaviour is evolving stationary figures with the purpose of finding the tallest and adding smell sensors to the mutation pool. In that example, if an individual gets a sensor during mutation, its existence does not improve searching progress.

2.1.4 Selection

The selection stage is responsible for the multiplication of good individuals. It is possible to manipulate the diversity of the resulting population by adjusting the number of best individuals that are chosen from the previous generation. The smaller number of top individuals is chosen the smaller diversity is occurring. *Higher selective pressure* is when there are properties giving big advantages to individuals that have them. When the pressure is too high it is possible to get stuck in the local optimum, contrary if the pressure is too low it is possible to make the evolutionary process ineffective [24]. There are various selection techniques. One of the most common is *roulette wheel technique* in which individuals have a chance to be selected proportional to the fitness they obtained. A different technique that is also very common is *k-tournament* where *k* members of the population are grouped in so-called tournaments and the best one is selected.

2.1.5 Fitness function

Based on a problem that is being solved suitable fitness function should be chosen. The most basic approach focuses on the fitness function that directly describes the problem. An example of this approach is calculating the heights of the figure when the target is to obtain the tallest possible figure. Using that fitness might cause the algorithm to trend in choosing the same direction which might not be optimal. There might be a case in which the algorithm chooses the simplest path to achieve good results causing getting stuck in the local optimum later on. To allow for bigger diversity inside population *niching* functions can be used. An example of such *niching* function is $fitness = dis \cdot fit$ where *fitness* is final fitness used by algorithm, *fit* is raw fitness describing the problem and *dis* is some dissimilarity measure calculated between individuals. This approach will promote bigger diversity inside the population as not only raw fitness value will be considered, but also how different is individual compare to others.

2.2 Framsticks

Framsticks is a three-dimensional simulation environment [28] whose main objective is to simulate life and evolution. The engine supports modelling objects (framsticks) with control systems. The software allows for different types of experiments such as optimization using *EAs*, evolution, simulation of different species and environments. There are various genetic representations [27] which have different possibilities and limitations.

2.2.1 Agents

Framsticks allows for the creation of models using one of the pre-defined genotype encodings or by adding new ones. Genotype encodings are languages that allow describing creatures. Different encodings have different effects on how evolution is progressing and what kind of outcome we can expect [26]. Each encoding has its own mutation and cross over operators which has a great impact on evolution. Creatures consist of two main parts, *physical body* that interacts with the surroundings and optionally with other bodies, and *brain* which is responsible for the behaviour of the creature. *Physical body* is composed of Parts that are connected using Joints. Both Parts and Joints have multiple parameters that impact creatures in the simulation. *Brain* of the creature is composed of neurons. There are multiple types of available neurons such as *sensors* (collect information from environment), *effectors* (muscles attached to parts that allow to move and rotate them) or signal processing neurons that process input and pass it to other neurons. By connecting all of this together it is possible to create creatures that can walk, swim, hunt [29] and a lot more.

2.3 Genetic representations

In *EA* each individual is coded using some form of representations of a solution for a given problem. Depending on the problem the representations might vary, for example in *TSP* problem it is enough to write a permutation that explains how the final path should look like. Other problems might require more complex representations, for example evolving tall structures [23] require a representation that allows describing 3D objects. For each problem, there is usually a number of representations that allow solving the problem in some way. Looking at 3D objects it can be seen that there are multiple representations capable of describing objects in three-dimensional space [39, 34, 30].

2.3.1 Genetic representations in Framsticks

In *Framsticks* environment there are different encodings allowing for various approaches to describing creatures. Below are listed some of them:

1. **f0/f0s** – it is so called “*Format Zero*”, every other encoding has to be transformable to this one as it is the direct representation of an agent. Each model described by *f0* representation consists of one or more objects. We can distinguish three types of objects such as:

- parts
- joints
- neurons
- neural connections

Each object has specific parameters allowing for accurate description. An example of such a genotype is:

```
//0
p:0.975, 1.09, -1.134, fr=1.317, ing=0.057, as=0.295
p:-0.199, 0.821, 0.376, fr=0.272, as=0.359
p:-0.972, 1.155, -1.023, fr=0.506, ing=0.252, as=0.88
p:0.107, 0.574, 0.362, fr=0.843, ing=0.044, as=0.07
p:-0.068, -0.531, -1.035, fr=1.311, ing=0.164, as=0.33
```

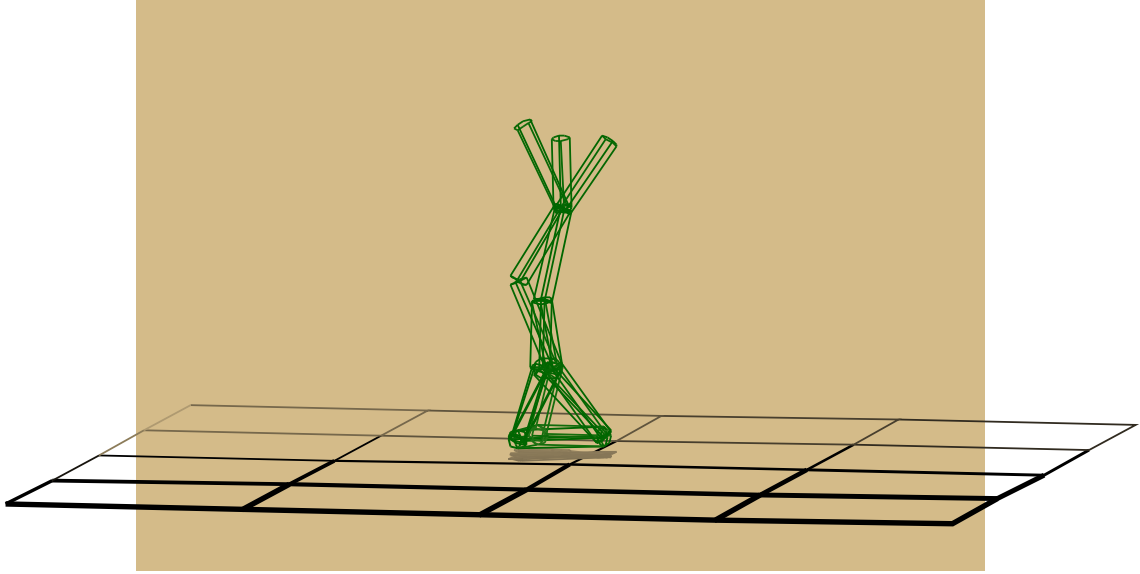


Figure 2.1: Example of a framstick encoded using the $f0$ representation.

```

p:0.67, 1.3566, 5.398, fr=0.28, as=0.071
p:0.85, 0.589, 5.112, fr=0.14, ing=0.25, as=0.20
p:0.16, 0.396, 3.720, fr=1.654, ing=0.33, as=0.31
p:0.757, -0.494, 5.102, fr=0.209, ing=0.19, as=0.18
p:-0.06, 1.311, 2.213, fr=0.461, ing=0.14, as=0.72
p:-0.182, 0.789, 1.791, fr=0.272, as=0.359
j:3, 10, stif=0.816, rotstif=0.819, stam=0.41
j:1, 10
j:1, 3
j:2, 4, rotstif=0.981, stam=0.06
j:3, 0, rotstif=0.811
j:2, 3, stif=0.99, rotstif=0.95, stam=0.19
j:2, 0, stam=0.38
j:0, 4, stif=0.96, rotstif=0.75, stam=0.23
j:2, 1
j:3, 4, stif=0.96, rotstif=0.75, stam=0.18
j:4, 1, stif=0.83, rotstif=0.97, stam=0.14
j:5, 7
j:6, 7, stif=0.91, rotstif=0.71, stam=0.23
j:10, 7
j:8, 7, stif=0.99, rotstif=0.87, stam=0.40
j:7, 9, stif=0.70, rotstif=0.95
j:9, 3, stif=0.76, rotstif=0.89, stam=0.31

```

Mutations change aspects such as location, connection or other parameter values of body and brain elements.

2. **f1** – called *recursive encoding*. It represents the graph as a tree. Subsequent elements are implicitly linked. Encoding is composed of characters such as:

- *X* – adds stick.

- (...) – adds branches.
- , – can be placed only in branches and is responsible for dividing.
- L, R, W, F – modifiers of lengths, rotation, weight, friction.
- [...] – generates neurons.
- ... – there are more modifiers not mentioned here.

An example of such genotype is **XX(X,LX(X,LX[1:1.0][-1:2.0]))**. Mutation in that encoding adds/removes modifiers, sticks, and adds/removes/modifies neural networks.

3. **f9** – is the “3D turtle” encoding that uses only six letters corresponding to words:

- *up* – “U”
- *down* – “D”
- *left* – “L”
- *right* – “R”
- *forth* – “F”
- *back* – “B”

This encoding doesn’t support control system (“*brain*”). An example of such genotype is **URRRDDLLU**. Mutation in that encoding adds or removes letters in a random location inside the string of characters.

2.3.2 Genotypes and phenotypes

In biology *genotype* is a set of genes of an organism. The characteristics of an organism are called *phenotype*. The same applies in *EA* but the role of genes take some kind of representation. *Phenotype* is influenced by both *genotype* and the environment that the organism or object is placed in. An example of such *phenotype* changes dependent on the environment can be seen in multiple animals such as:

- Color of flamingos is depended on the food they eat. If they eat food that contains a lot of pigment called carotenoids they can be pink. Without food containing that pigment, their feathers are white.
- Siamese cats have dark fur when living in a cold environment, but white fur while living in a warm environment.
- Flowers of *Hydrangea macrophylla* plant have different colors depending on soil *pH*.

In *EAs*, the *genotype-phenotype* relation varies based on the problem. For example in some cases such as *TSP phenotype* is directly related to *genotype* and there are no deviations between them. Taking as an example *Framsticks* and the available representation over there it can be seen that one model can be represented using different encoding and each encoding has a different correlation to resulted phenotype. There is also the impact of the environment based on the problem that is solved. Comparing the same *genotype* in two different environments can produce two totally different *phenotypes*. The same creature placed on the ground and in the water can behave with different characteristics. Changing some parameters such as gravity can also lead to a change in behaviour. There is also the aspect of the impact that change in *genotype* has to the result in *phenotype*. Analyzing the evolution of moving structures using *f1* encoding it can be seen

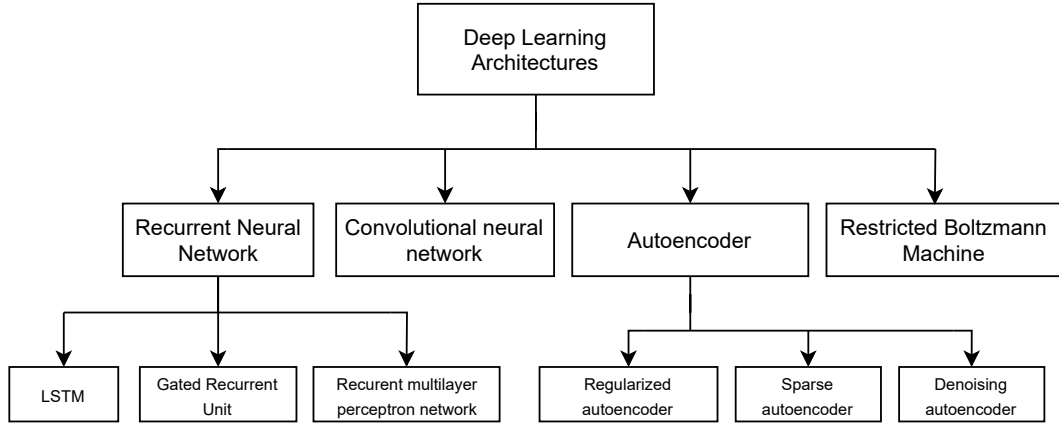


Figure 2.2: Types of Deep Learning Models.

that sometimes changes that at first might look as important and big, don't produce any changes in *phenotype*. On the other side, some small changes but aimed at the correct aspect of *genotype* can produce huge changes in *phenotype*. A good example of it changes in neurons, a small change can make that structure that was not moving at first will start rolling, or a structure that had synchronized steps will become unable to make the next step.

2.3.3 Impact of encoding

Genotype represented as *genetic encodings* describes the solution and has an impact how the evolutionary process will progress. There are multiple implications of how the encoding can impact the searching process of *EA*:

- Higher level encoding might introduce bias in the produced solutions [26].
- Some results might not be possible to be produced using given representation (e.g., moving structures using *f9 encoding*).
- Moving through searched space is not always continuous (e.g., adding new part) [30].
- Encoding with their *Genetic operators* has an impact on which solutions are neighbour to each other which can greatly change the topology of searching space [30].

Because of these implications, different *encodings* can behave differently for a given problem. It is shown that the quality of the result can vary based on the encoding, that is why choosing the correct one has a huge impact on the results [30]. Nonetheless choosing the right one is also a hard problem because of many characteristics that need to be considered, and even then it is hard to understand the searching space for a given problem without experiments.

2.4 Deep Learning

Deep learning is a part of *machine learning* that focuses on teaching computers how to perform tasks that are easy to perform but hard to describe by humans. The tasks are solved by teaching computers dependencies, by giving examples thanks to which computers learn the concept of a problem from simpler concepts [16].

The term *Deep learning (DL)* covers various different architectures. Some of most popular architectures are visible in Fig. 2.2. Each architecture has its own characteristics and strong

points. *Recurrent Neural Network* differs from the rest by using memory, obtained by recurrent connections inside the network. While *Convolutional neural networks* perform well with image recognition and features extraction. Currently there are multiple solutions that combine different *deep learning* architectures to create a new one. The main aspect that *DL* architectures have in common is that they have multiple layers. Because of that *DL* are very good at adjusting to the problem that is being solved. Unfortunately it can also cause problems such as *over-fitting*. With so many layers of abstraction, network has a lot of space to over-fit which can result in bad generalization. In recent years due to incredible achievements, *deep Learning* is in the center of attention in multiple fields of science. The biggest progress can be seen in fields such as:

- *Computer vision* – with *GAN-s* generating realistic pictures that never existed [17] and many other achievements in the field [36, 42, 9].
- *Voice* – with *WaveNet* generating speech that mimics human voice [40].
- *Text* – with *google translate* achieving better accuracy with suggesting translations that the users were looking for [43].

There are multiple factors responsible for the success of *deep learning* such as improvements in computer hardware and common digitization resulting in rapidly expanding the availability of data sets for various fields. Thanks to these factors *deep learning* models expand in size and are able to learn new and harder tasks with better and better effectiveness. Currently there are multiple powerful frameworks that allow easy testing models for a given problem:

- *TensorFlow*,
- *Pytorch*,
- *scikit-learn*,
- *Spark ML*.

2.4.1 Autoencoders

An *autoencoder* is a neural network that is trying to learn the representation of given input in order to produce its copy on the output. The beginning of autoencoders date to the 1980s with dimensional reduction and features learning [16]. General structure of an autoencoder contains an input $\mathbf{x}$ that is mapped on internal representation $\mathbf{h}$ using encoder. From representation $\mathbf{h}$ decoder produces reconstruction $\mathbf{r}$ that is suppose to be as close to original $\mathbf{x}$ as possible. The difference between $\mathbf{x}$ and $\mathbf{r}$ is called *reconstruction error*. Example of that structure is visible in Fig. 2.3.

Main uses of autoencoders are:

- Dimensionality reduction – it is proven that high-dimensional data can be reduced to low-dimensional codes [19].
- Anomaly detection – Model learns how to reconstruct data by finding and encoding the most important features. If the features of the new example are to differ from what the model expected, the model will fail to reconstruct the given example.
- Noise reduction – It is possible to reduce or even remove noise from images using autoencoder architectures [7].

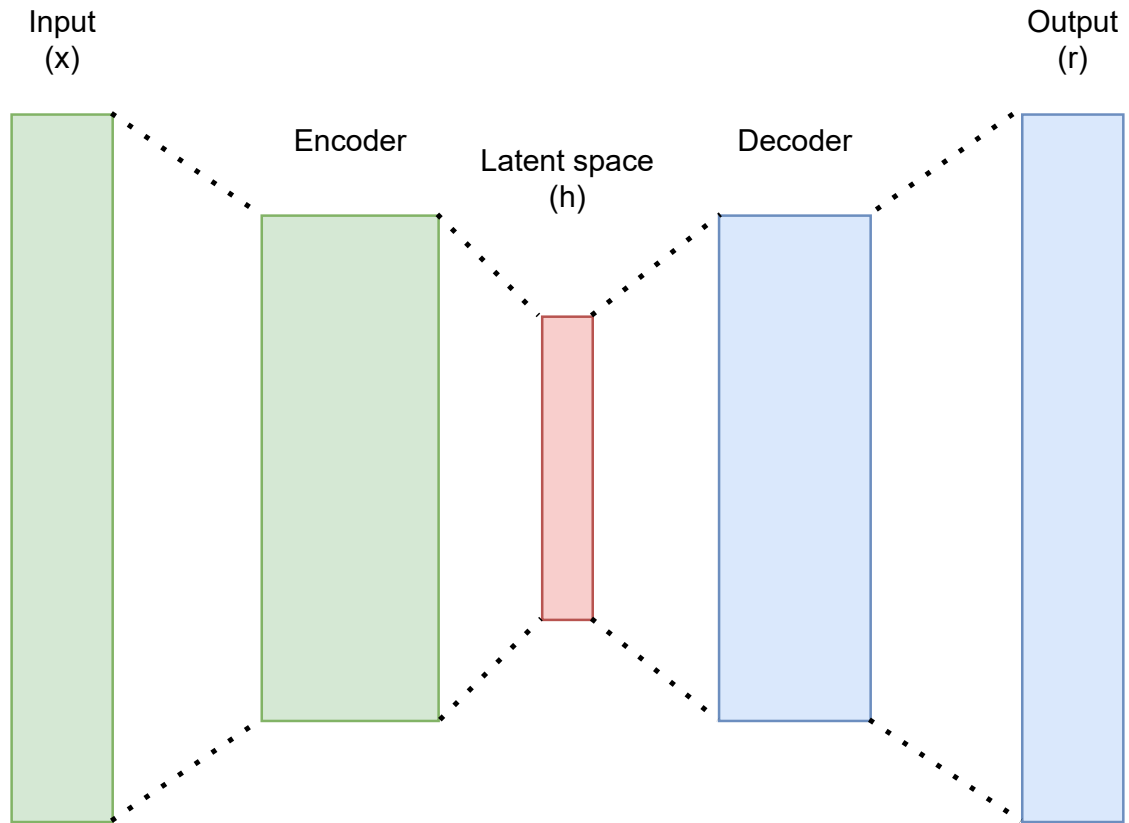


Figure 2.3: Example of basic autoencoder architecture.

- Feature learning – Allows discovering features that will be useful for future machine learning problem.
- Image compression – Using dimensional reduction aspect it is possible to compress images [38].
- Learning representation for evolutionary search [33].

Many versions of autoencoders have been created over the years, below are some of them:

- *Undercomplete autoencoders* – *autoencoder* is called undercomplete when $\mathbf{h}$ has smaller dimension than $\mathbf{x}$. Reducing dimensions of $\mathbf{h}$ is made with intention to force *autoencoder* to learn useful properties about $\mathbf{x}$. If number of dimensions of $\mathbf{h}$ is too big, *autoencoder* fails to extract any properties. If the number of dimensions of $\mathbf{h}$ is equal or greater compared to the number dimensions of $\mathbf{x}$ autoencoder can just copy the input to the output without extracting anything.
- *Regularized autoencoders* – allows autoencoder for $\mathbf{h}$ to have a number of dimensions equal or higher than $\mathbf{x}$ by using a loss function that encourages other properties than just copying input on the output. Examples of such properties are denoising or sparsity of the representation.
- *Sparse autoencoders* – loss in *sparse autoencoder* except reconstruction loss contains also sparsity penalty on the layer $\mathbf{h}$. The main role of *sparse autoencoders* is to learn the features of given input. Features learned given such *autoencoder* can be later used in other tasks or be a part of a bigger model.

- *Denoising autoencoders* – changes the reconstruction loss so his target is to calculate [16]:

$$L(x, g(f(\bar{x})))$$

where $\bar{x}$ is input with noise, g is decoder and f is encoder. This way we force *autoencoder* to repair input rather than just copy it.

- *Variational autoencoders* – is a generative model that learns stochastic mappings of the input space $\mathbf{x}$. To achieve that, the autoencoder regularizes training to avoid overfitting and to ensure good latent space properties [21].

2.4.2 Graph neural networks

Lots of solutions requires representation that can contain complicated relation information. One of the way to describe these is to use graph data representation which can be seen in different applications such as social networks, modeling physics, proteins or images. As graph structures become more and more common demand for learning models also rises. As an answer came *Graph Neural Networks (GNNs)* [44, 14, 18]. *GNNs* are type of neural networks that uses *message passing* between nodes of graph to catch specific dependencies that are located inside. Its beginnings *GNNs* date for late 1990 with models aimed at directed acyclic graphs [37]. In later years *GNNs* got enriched by *Recurent Neural Networks* and *Feed Forward Neural Networks* [46]. Further development of *GNNs* got attention as this approach showed great potential with non euclidean data [5] which is a bit problem for *convolutional neural network CNNs*. Below is the list of main elements that are needed for graph operations in *GNN* implementations:

- n – Number of nodes.
- f – Size of node attributes.
- A – Adjacency matrix that represents the connections between nodes. Square matrix of the size $n \times n$ where the n represents the number of nodes in graph.
- X – Feature matrix of the size $x \times f$ where n is the number of nodes in graph and f is the number of features representing the graph and x_i are node attributes corresponding to i -th node.
- E – Edge attributes where $e_{i \rightarrow j}$ are an edge attributes between node i and j .
- Y – label of a graph of list of labels for nodes if they exist.
- D – Degree matrix.
- W, V – Trainable weight matrices.
- b – Trainable bias vector.

These are not all used properties of graphs but most of the implementations require A and X as the bare minimum. In graph learning there can be distinguished three levels on which learning is performed:

1. *Node-level* – task focused on searching nodes information. Depending on task node can represent class for classification problem, value for regression problem or nodes can be cluster in clustering problems.

2. *Edge-level* – this tasks aim ad predicting edge types or whether they exist or not.
3. *Graph-level* – this task types aim at learning graph representation as whole. In classification problem the whole graph has some class.

There are multiple implementations with of *GNNs*, here are some examples with them:

- *MP* – MessagePassing [15] which computes

$$\mathbf{x}'_i = \gamma(x_i, \square_{j \in \mathcal{N}(i)} \phi(\mathbf{x}_i, \mathbf{x}_j, \mathbf{e}_{j \rightarrow i}))$$

where:

- γ is differential update function,
 - ϕ is differential message function,
 - $\square$ is a permutation-invariant function to aggregate messages.
- *GCNconv* – Graph convolutional layer [22] which computes

$$X' = \hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2} XW + b$$

where:

- $\hat{A} = A + I$ is adjacency matrix with added self loops,
- $\hat{D}$ is degree matrix.

GCN-s is the baseline for graph convolution-s and uses graph Fourier transform.

- *GCSCconv* – Graph convolutional layer with trainable skip connection. This layer computes:

$$Z' = D^{-1/2} A D^{-1/2} XW_1 + XW_2 + b$$

where A does not have self-loops.

- *GATconv* – Graph attention convolutional layer [41] which computes

$$X' = \alpha XW + b$$

where

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(a^T[(XW)_i || (XW)_j]))}{\sum_{k \in \mathcal{N}(i) \cup \{i\}} \exp(\text{LeakyReLU}(a^T[(XW)_i || (XW)_k]))}$$

and $a \in \mathbb{R}^{2F'}$.

GATconv uses attentional layers that allow attending into features of neighbours. By using an attention mechanism, the model is able to assign different weights to nodes in the same neighbourhood which allows for a bigger model capacity. Authors of the architecture also mention its high efficiency compared to the baseline *GCNs* architecture [41].

- *ARMAconv* – Auto-Regressive Moving Average convolutional layer [4] which computes

$$X' = \frac{1}{K} \sum_{k=1}^K \bar{X}_k^{(T)}$$

where K is the order of ARMA_K filter and where

$$\bar{X}_k^{(t+1)} = \sigma(\tilde{A} \bar{X}^{(t)} W^{(t)} + X V^{(t)})$$

is a recursive approximation of ARMA_1 filter where $\bar{X}^{(0)} = X$ and

$$\tilde{A} = D^{-1/2} A D^{-1/2}$$

The strong point of *ARMAconv* is its use of *auto-regressive moving average* filter which, compared to polynomial solutions, is more resistant to noises and, as the authors mentioned, is good at grasping global graph structure [4].

Thanks to the usage of *GNNs*, a new approach to different graph-oriented problems emerged. Some of the problems that found solution in *GNNs* are:

- *Graph and Node Classification* – classification on graph or node level.
- *Link prediction* – prediction of connections between nodes of given graph.
- *Graph Embedding* – keeps properties of a graph in vector representations.
- *Graph Generations* – using artificially created representation of graphs to create new ones.
- *Graph Clustering* – dividing graphs nodes in to clusters.

2.5 Latent representation

Latent space (LS) is a created representation of some existing problem in an abstract multi-dimensional space. *LS* is created by extracting features from the given problem. Based on these features machine learning model creates its own representation of problems. By creating *LS* it is possible to discover unapparent aspects of a problem. The space is created by feeding the model/algorithm a set of examples that represent the problem. The created latent space can be very different from the original space that humans perceive as the computer has only a limited set of examples and no other references.

Let us say that the problem is to classify an animal. Data sets consist of 4 different species with proper descriptions that allow for differences between them. By teaching the model on that data, the model should be able to group given animals in 4 groups by learning their respective features based on given descriptions. An example of such distribution can be seen in Fig. 2.4. In that artificially created space, objects which are similar (have similar features) are placed close to each other, while the objects that are different (different features) should be farther from each other. In a perfect scenario, there will be a correlation between the difference of objects and the distance between them in the latent space.

Most of the times, the dimensionality of *LS* is set to be lower than dimensionality of the examples. By lowering the size of dimensions the *LS* is a compressed version of original problem which means that model is required to create a representation from the most important features and reject non-important ones and noise. There are multiple architectures operating on latent space, some of the examples are:

- *Autoencoders*,
- *Word embeddings*,
- *Topic modeling*,
- *GANs*.



Figure 2.4: Visualisation of two dimensional latent space with 4 types of classes. Each class is represented by different color.

Chapter 3

The proposed approach

Both genetic representations and operators are created with intention of allowing an easier search for given solutions. Choosing correct representation and adjusting operators that are available for evolutionary algorithms might allow for better control of searching through the space of possible solutions. Examples of such choices might lead to reduced searched space by reducing the number of accessible solutions or allow for a smoother walkthrough solution space. Unfortunately, it is often hard to find perfect representation for a problem, as there are multiple variables to be considered such as:

1. Readability of a problem from the human perspective.
2. Possible fitness limitations as some representations may not be able to cover the whole space of a problem.
3. Complexity of representation and its transfer to fitness landscape shape. It is best if the fitness landscape is smooth and changes made in *genotype* correspond to changes in *phenotype*.

The goal of this thesis is to present an approach to generate latent representation of given problem which can be further used to develop better solutions. Presented approach uses graph representation of given problem which is then learned by *auto-encoder* architecture. This solution aims at smoothing searched landscape, and with this enable *EA* to find better solutions.

3.1 Architecture

Presented architecture consists of few main elements as shown in Fig. 3.1:

- *Environment*
- *Data set*
- *Autoencoder* that consists of three sub elements
 - Encoder
 - Decoder of A'
 - Decoder of X'
- *Locality loss manager*
- Final processing of output

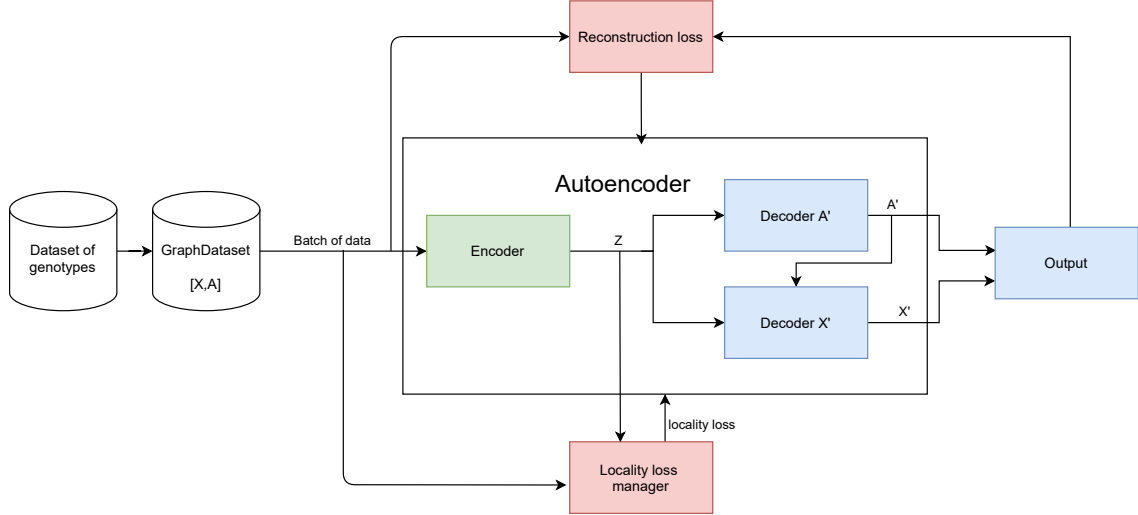


Figure 3.1: The diagram of proposed architecture.

3.1.1 Environment

For the purpose of obtaining data and performing optimization experiments *Framstick environment* was used. The listed features have been used:

- Evaluation of genotypes,
- Dissimilarity measure between genotypes,
- Mutation operations for $f0$ representation,
- Conversion of $f0$ representation in to a model.

3.1.2 Graph data-set

Graph Data-set module is responsible for translating the original data-set of genotypes into a data-set consisting of graphs that have X (feature matrix) and A (Adjacency matrix). In this thesis input data-set consisted of genotypes represented using $f0$ representation. Thanks to properties of $f0$ representation it is very easy to translate it to graph representation and so each graph in the data-set represents one individual.

Translation goes as follows:

- Size of A is $n \times n$ where n is maximum number of parts possible.
- Size of X is $n \times f$ where f is number of features representing each part.
- Number of parts in individual is represented by p and $p \leq n$.
- p rows in X are filled with f considered features, $n - p$ rows are filled with -1 .
- To make sure that no feature achieves value -1 , all features lower than 0 got shifted.
- Matrix A is filled with 0 and 1, where 1 is in places corresponding to joint connections which means that $n - p$ rows are filled with 0.

In this approach, neurons are not considered, but there are two possible ideas should neurons be needed in the future.

The first idea assumes that neurons are presented as parts. In that case, n must be expanded by the total number of extra neurons and f must be expanded by the number of features required to express neurons. Entries in X that correspond to parts would have features of parts filled and features of neurons set to some non-existent value while entries corresponding to neurons would have features of parts set to non-existent value. In this example, it is possible to connect each neuron individually with other neurons and parts, but the size of X and A rises linearly to the number of neurons.

The second idea assumes that each neuron is connected to the part and every aspect of the neuron is presented as a feature of that part. In that case, n doesn't change and only f must be expanded by the number of features necessary to represent neurons. Parts that don't have neurons have features corresponding to them filled with non-existent value. In this idea, there is a limitation in the form of the required relation part–neuron.

3.1.3 Autoencoder

The featured autoencoder is responsible for connecting three networks: an encoder and two decoders. Optionally *Custom loss manager* can be added to affect the smoothness of latent landscape. For the purpose of implementing mentioned networks, custom layers were defined as shown in Fig. 3.2. In all layers except penultimate layer in *Encoder Relu* activation is used. For custom convolutional layer there are 4 possible choices defined:

- *GCSconv*
- *GCNconv*
- *ARMAconv*
- *GATconv*

Description of these layers was mentioned in Sect. 2.4.2. In this paper two main approaches to autoencoder architecture are tested:

1. *Graph Auto Encoder (GAE)* – approach featuring traditional autoencoder architecture using *GNN* layers.
2. *Variational Auto Encoder(VGAE)* – *Variational* version of autoencoder with *GNN* layers.

Both architectures were tested using the same parameters and the same convolution layers.

Encoder GAE

Encoder network takes as input two matrices A and X described in Sect. 3.1.2. The structure of *Encoder* is visible on Fig. 3.3. First part of *Encoder* consists of *Conv layers*, minimum number of *Conv layers* is 1 and maximum is 4. Each *Conv layer* uses as input original A and output of previous *Conv layer*, exception from that is first *Conv layer* which as input takes original X . Then there are *Dense layers* with a number of layers in the range from 2 to 7 The last layer is the size of latent space.

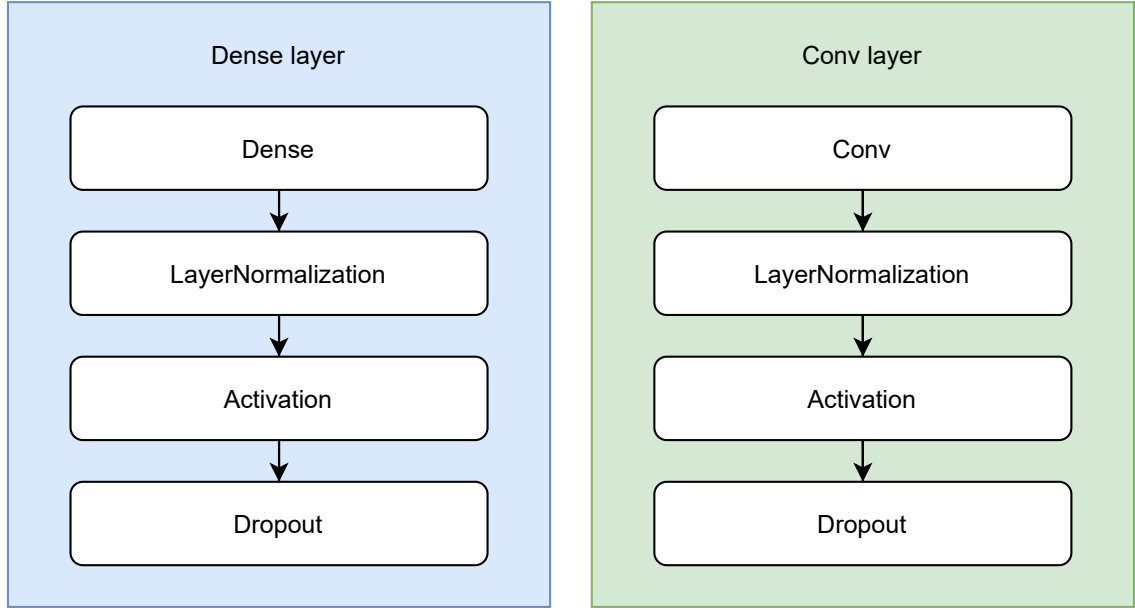
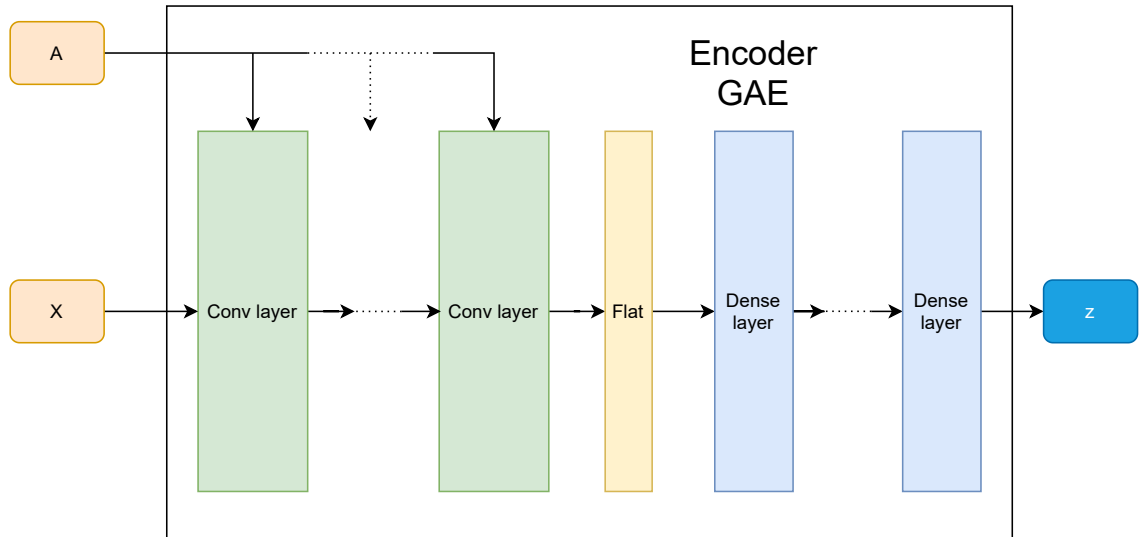


Figure 3.2: Custom layers defined for the purpose of presented architecture.

Figure 3.3: The diagram presenting architecture of Encoder for *GAE* variant.

Encoder VGAE

A variational version of Encoder extends the basic version with two parallel dense layers that produce z_{mean} and $z_{var \log}$ which are used in sampling and to calculate $kl \text{ loss}$. Formula by which sampling is done looks as follows:

$$z = z_{mean} + e^{(0.5 * z_{log-var})} \cdot \epsilon$$

where:

- e – euler number,
- ϵ – random matrix of normal distribution.

Visualisation of *Variational Encoder* is shown in Fig. 3.4.

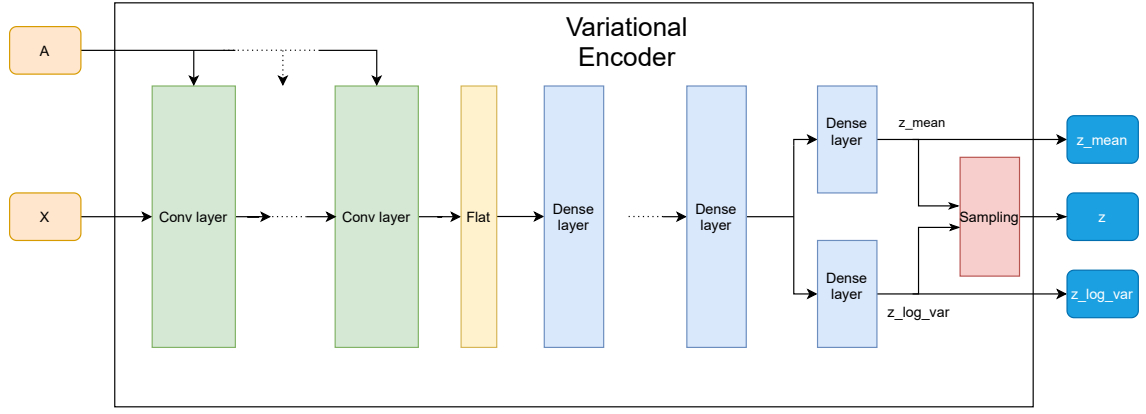
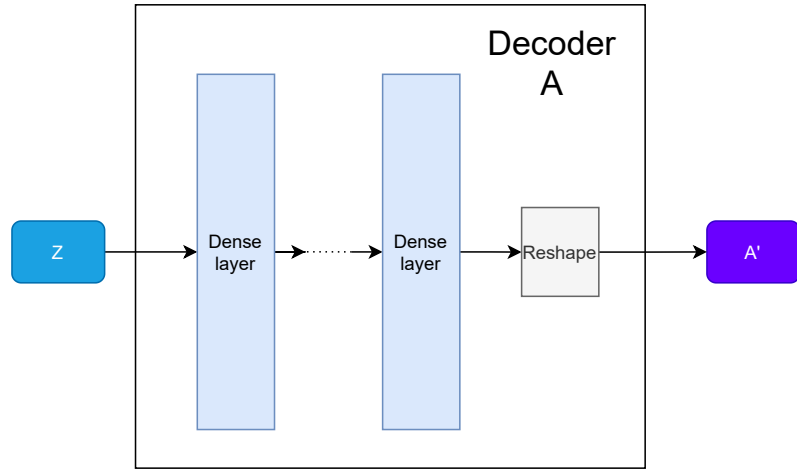
Figure 3.4: The diagram presenting architecture of Encoder for *VGAE* variant.

Figure 3.5: The diagram presenting architecture of Decoder A.

Decoder of A'

Decoder of A' consists of only *Dense layers*. The number of *Dense layers* is between 1 and 5. Last layers has *Sigmoid* activation function as matrix A must contain values 0 and 1. Final rounding to achieve only 1 and 0 is done in post-processing outside the network. The final step of the decoder is reshaping the matrix to the size of $n \times n$ where n is the maximum number of allowed parts. Visualisation of *Decoder A* is shown in Fig. 3.5.

Decoder of X'

Decoder of matrix X uses do reconstruction A' generated previously by *Decoder A'*. Similar as in *Encoder Decoder X'* consists of *Conv layers* and *Dense layers*. The flow in *Decoder X* presents as follow:

1. z goes through dense layer to obtain the size of $latent_size \times n$.
2. For next i (where $i \in [1, 2]$) *Conv layers* layer takes output of previous layer and A' .
3. flatten of *Conv layers* output
4. i *Dense layers*, where $i \in [1, 7]$. Last layer has size of $n \cdot f$
5. *Reshape* in to matrix X' of size $n \times f$ and subtract 1 as *Relu* cant obtain -1 which is required as fill.

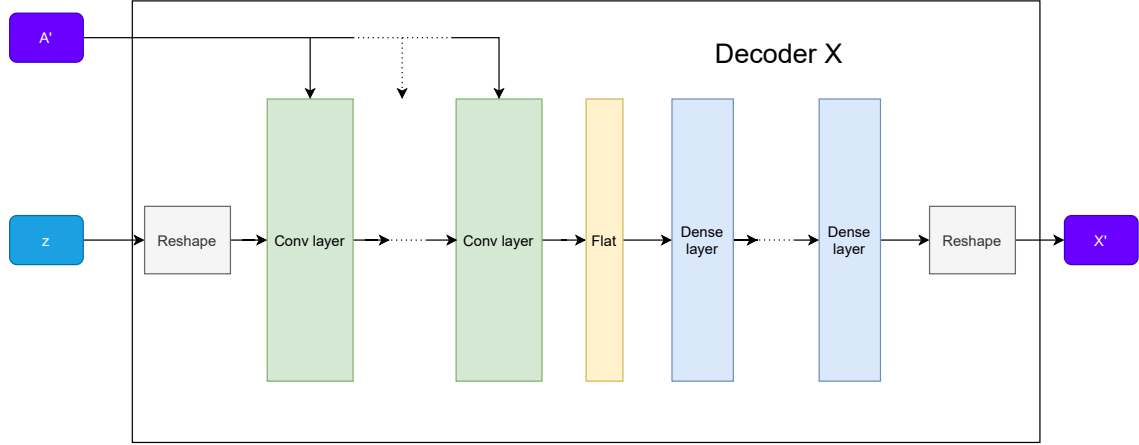


Figure 3.6: The diagram presenting architecture of Decoder X.

Visualisation of mentioned flow is shown in Fig. 3.6.

Reconstruction Loss

Reconstruction loss can be defined by (3.1).

$$reconstruction_loss = MSE(A, A') \cdot w + MSE(X, X')$$

where:

- *Mean Square Error* – $MSE(x, x') = \frac{1}{n} \sum_{i=0}^n (x_i - \hat{x}_i)^2$,
- A – adjacency matrix and A' is reconstructed adjacency matrix,
- X – features matrix and X' is reconstructed features matrix,
- w – weight increasing importance of reconstruction loss of A .

(3.1)

For purpose of experiments, w was set to 1000. Setting w allowed for a similar impact of both reconstruction loss X' and A' , which resulted in faster learning progress. Variational version of autoencoder uses additional *kl loss* which is described by (3.2).

$$kl_loss = reduce_mean(reduce_sum(-0.5 \cdot (1 + z_log_var - square(z_mean) - \exp(z_log_var))))$$

where:

- z_mean and z_log_var are produced by variational encoder.

(3.2)

3.1.4 Locality loss manager

Using only base reconstruction loss, autoencoder will learn how to reconstruct data, but latent space landscape can be of little value. To impact the shape of latent space *locality loss* is introduced. The purpose of *locality loss* is to encourage network to encode similar genotypes close to each other

in *latent space*. Locality loss used in the thesis is calculated using:

$$locality_loss = \epsilon \cdot (1 - corr)$$

where:

- *corr* – Spearman correlation between mean distance in latent space and chosen smoothed property,
- ϵ adjusts importance of locality loss.

In this work there are 3 aspects examined.

- *parts number* – calculates the correlation between the number of parts and mean distance to other genotypes in batch.
- *fitness* – calculates the correlation between selected fitness and mean distance to other genotypes in batch.
- *dissimilarity* – calculates the correlation between mean dissimilarity and mean distance to other genotypes in batch.

3.1.5 Post processing

The output of autoencoder can be always converted into corresponding to it *f0* representation. The problem is that not every representation is valid for the used environment. There are multiple problems with genotype, but only some are prohibited as they cannot be evaluated. Forbidden occurrences are:

- Too long joints between parts, default value in *Framsticks* is 2.
- Sub groups of parts that are not connected with the rest.
- Joints of length 0 (impossible to simulate because of an undefined 3D orientation).

To address these problems, post-processing is performed. The algorithm repairing isolated parts is shown as Alg. 2. As an input algorithm takes a list of parts *p* and a list of joints *j*. Mentioned there *group()* creates a list of all subgroups located in graph described by *p* and *j*. Function *connections()* is responsible for finding all connections between parts of two different subgroups passed to the function. By having all minimal connections between each subgroup in the graph, it is possible to create the shortest joints between mentioned subgroups.

Removing too long joints is done by moving parts with lower connections towards the centre of the joint, similar to the repair procedure implemented natively in *f0* genetic operators. The algorithm describing it is shown as Alg. 3. It is possible to get stuck using this algorithm, as there might be a situation in which moving part to reduce the length of one joint makes the other joint too long. To reduce the possibility of that occurring, parts with a lower number of connections are chosen, but deadlock may still emerge. Because of that deadlock, a counter with the maximum number of iterations is introduced. If the algorithm can not remove too long joints during *max_it* iterations, the individual is treated as invalid.

The last problem of joints having a length equal to 0 is simply solved by adding very small ϵ . Except for situations where deadlock during removing too long joints occurs, all individuals are repairable. If a genotype cannot be made fully valid by employing these procedures, it is considered invalid and is discarded.

```

Data:  $\text{len}(\text{parts}) \geq 0$ 
          $\text{len}(\text{joints}) \geq 0$ 
Result:  $\text{parts}, \text{joints}$ 
 $\text{groups} \leftarrow \text{group}(\text{parts}, \text{joints});$ 
 $\text{possible\_connections} \leftarrow [];$ 
while  $\text{len}(\text{groups}) \geq 0$  do
  for  $i$  in  $\text{range len}(\text{groups})-1$  do
    for  $j$  in  $\text{range}(i+1, \text{len}(\text{groups}))$  do
       $\text{possible\_connections}$  append  $\text{min}(\text{connections}(\text{groups}[i], \text{groups}[j]));$ 
    end
  end
   $\text{connection}, i, j \leftarrow \text{min}(\text{possible\_connections});$ 
   $\text{joints} \leftarrow \text{joints} + \text{connection};$ 
   $\text{groups}[i] \leftarrow \text{merge}(\text{groups}[i], \text{groups}[j]);$ 
   $\text{remove}(\text{groups}[j]);$ 
end

```

Algorithm 2: Matching isolated parts.

```

Data:  $\text{len}(\text{parts}) \geq 0$ 
          $\text{len}(\text{joints}) \geq 0$ 
          $\text{max\_it} \geq 0$ 
          $\text{max\_len} \geq 0$ 
Result:  $p, j, \text{counter}$ 
 $\text{too\_long\_joints} \leftarrow \text{find\_too\_long}(\text{parts}, \text{joints});$ 
 $\text{counter} \leftarrow 0;$ 
while  $\text{too\_long\_joints} \geq 0$  and  $\text{counter} \leq \text{max\_it}$  do
  for  $j$  in  $\text{too\_long\_joints}$  do
     $p1, p2 \leftarrow j;$ 
     $p\_less, p\_more \leftarrow \text{less\_connections}(p1, p2, \text{joints});$ 
     $p\_less \leftarrow \text{move\_to\_center}(p\_less, p\_more, \text{max\_len})$ 
  end
   $\text{counter} \leftarrow \text{counter} + 1;$ 
   $\text{too\_long\_joints} \leftarrow \text{find\_too\_long}(\text{parts}, \text{joints});$ 
end

```

Algorithm 3: Removing too long joints.

3.2 Data

For training purposes, two data sets were used. Both data set are constructed from a collection of individuals that were generated using evolution in *Framsticks* environment. The first data set (Velocity data set) consists of 25039 examples. Individuals got obtained from *hall of fame* of different experiments that were aiming at optimizing velocity. Because of the characteristics of the velocity optimization problem, most genotypes are small and don't reach a high vertical position. Genotypes are represented using *f0* and *f1* representations. Statistics describing data sets are obtained after translating genotypes into graph representation which will be later used for training purposes. This operation can cause some information loss as considered graph representation does not include all properties of original encodings *f0* and *f1*. Because of that, genotypes used in experiments are simplified and differ with properties from original genotypes obtained using evolutionary algorithms. The maximum vertical position (*vertpos*) in the first data set is 1.1357 and its average is around 0.049. The number of parts per individual inside the data sets varies from 2 to 15 with the average around 7. The distribution of the number of parts and joints per individual is visible in Fig. 3.7. More details is presented in Table 3.1.

The second data set (*vertpos data set*) consists of 10504 examples obtained from evolution

	vertpos	number of parts	number of joints
mean	0.049078	7.181976	7.816807
std	0.107883	3.595919	6.201249
min	-0.049551	2	1
25%	-0.015000	4	3
50%	0.004503	6	6
75%	0.068423	10	11
max	1.135704	15	30

Table 3.1: Table describing features of first data set.

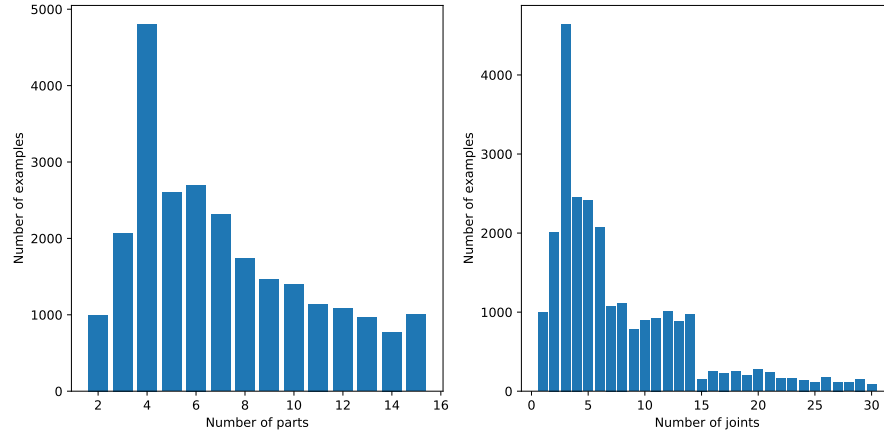


Figure 3.7: Distribution of the number of parts and joints in velocity data set (evolved for highest velocity).

	vertpos	number of parts	number of joints
mean	1.268434	10.096820	11.068926
std	0.821135	3.679957	6.260979
min	-0.010000	1	1
25%	0.485066	8	7
50%	1.353009	11	10
75%	2.070922	13	14
max	2.724392	15	30

Table 3.2: Table describing features of second data set.

directed at optimizing vertical position. Individuals in this data set have a higher average vertical position which is around 1.27. Maximum *vertpos* among the examples reaches 2.7243. The average number of parts and joints is also higher compared to the first data set. Distribution for the number of joints and parts is visible in Fig 3.8. More detailed description of data set is presented in Table 3.2.

By comparing Figs. 3.7 and 3.8 it can be seen that distribution of both number of parts and joints are different. *Vertpos* data set contains more genotypes with high numbers of parts and joints. The reason for that is vertical position fitness favors bigger, more “bushy” individuals.

3.3 Conducted experiments

This section describes conducted experiments. All experiments can be divided into three sections. The first set of experiments was aimed at testing autoencoder architectures and finding parameters with best reconstructional properties. Reconstruction loss was measured using *Mean Square Error*

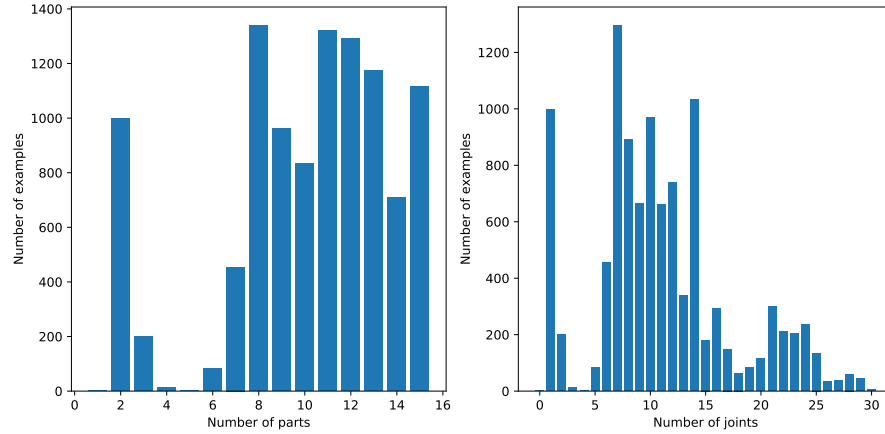


Figure 3.8: Distribution of the number of parts and joints in vertpos data set (evolved for highest vertical position).

defined as it is described in Sect. 3.1.3. The next part of the experiments focused on smoothing. To achieve that custom made locality loss functions were used. Description of locality loss was presented in Sect. 3.1.4. The purpose of the last part of the experiments was to test the best models in the evolution process. During evolution autoencoder is role was to perform mutation on chosen members of the population.

For every experiment presented the following settings for data representations have been applied.

- Adjacency matrix A has the size of 15×15 which allows maximum number of 15 parts.
- Features matrix X has the size of 15×3 .
- Three features represented in X correspond to three coordinates of given part.

Given the above restriction, it is possible only to represent static genotypes. It could be possible to add more features to matrix X and allow graph representation to reconstruct more complicated structures. But the aim was to get a better understanding of the problem by simplifying and reducing the layers of abstraction. To do so, only coordinates were chosen as features.

3.3.1 Architecture development

For the purpose of finding architecture giving the best results, multiple tests were performed with different parameters. All tests in this part were performed on the velocity data set. For learning purpose data sets was randomly split into training and test sets with the proportion of 0.8 for training and 0.2 for testing. The training was performed on a custom training loop based on *Keras* architecture. In experiments, *Adam* optimizer was used with decaying learning rate. Learning rate starts at 0.01 and decays by $(\frac{p}{10})^{0.7}$ where p is the number of epoch. Value of learning rate and its decay was selected based on experimental results as with lower learning rate architecture had problems with learning and without decay, the loss was too unstable. Experiments were conducted both on standard and variational versions of the architecture.

3.3.2 Locality loss experiments

Architectures that presented the best results were chosen for locality loss experiments. Locality loss experiments can be divided into three parts based on what in which direction they try to

smooth autoencoders latent space:

- *Parts number locality loss* – it aims to ensure that genotypes close to each other in latent space have a similar number of parts.
- *Similarity locality loss* – purpose of that loss is to keep similar individuals close to each other. To calculate similarity, a graph-based measure from Framsticks [25] was used. Distance between two given graphs is calculated as follows:
 1. Vertex corresponds to a part, if a part has neurons then they are attached to the vertex.
 2. Matching the most similar verices from two given graphs and for each calculates weighted sum of:
 - geometric distance between vertices,
 - difference of vertex degree,
 - difference in neuron number.
 3. Adding penalty for additional vertices that do not have a match.

The reason for choosing this measure is its similarity in representation used by presented architecture and its availability and compatibility as the implementation was ready in Framsticks SDK.

- *Fitness locality loss* – aims at preserving close to each other in latent space individuals similar in the context of fitness. For experiment purpose as fitness *vertical position* is taken. Obtaining fitness for model learning experiments is performed once during loading data. Each individual is converted in to X and A , then converted into $f0$ representation again before being evaluated by the Framsticks environment. After that, the data set consists X , A and y which in that case is vertical position. The operation that is needed to evaluate individuals after they are deprived of some properties that are not considered during an experiment.

3.3.3 Evolutionary experiments

This part of the experiments focused on studying the evolutionary possibilities of learned models. For simulation and evaluation purposes Framsticks environment was used. Framsticks SDK is written in cpp, while the proposed architecture uses python implementation. To communicate with environment python interface [1] was used.

During evolutionary experiments, the role of the model was to mutate selected individuals from the population. Mutation process can be described as follows:

$$i' = dec(enc(i) + r)$$

where i' is mutated individual, and r is random vector of size latent space. The range of numbers for r was selected based on experimental results from exploring latent space.

Framstick environment used default setting with the change to:

- Energy0:5000.0 – lowers base energy limit to shorten simulation as vertpos evolution doesn't require long simulations,
- randinit:0.0 – reduces non-determinism,

- `bnoise_struct:0.01` – prevents evolution from abusing situations as stacking sticks perfectly on top of each other,
- `perfperiod:2` – increase sampling accuracy.

This version of architecture doesn't examine cross operation, because of this mutation probability is set to 1.0 while cross probability is set to 0.0. To determine which individuals chose to mutate, tournament selection with $k=3$ is used. As a baseline to conducted experiments, evolution using Framsticks was also conducted. Baseline evolution used the same setting and f0 representation. All evolutions were performed over 2000 generations with a population of size 100. As mutation is equal to 1.0 and there is not certainty that the last population will contain the best individuals, the Hall of fame of size 100 is used, to keep 100 best individuals created during evolution.

3.4 Description of landscape analysis methods

This section describes methods used to analyze the properties of trained models.

3.4.1 Mutation impact

During the evolution process, the algorithm chooses promising individuals with good fitness values and applies mutation or crossing operations to them. If the mutation is too big then there is a higher chance that a new individual will lose properties that were giving his predecessor good fitness. When the mutation is too small, there might be no impact on new individuals and the evolutionary step will be a waste of resources. To analyze how strong the mutation operator should be, the following experiment was conducted. For the given model two sets of 1000 points were prepared. The first set consisted of 1000 random points sampled randomly from latent space, the second set consisted of 1000 random examples from data set that were converted into latent space using the encoder. Points were chosen randomly so inside the sample there can be both examples from the train set and test set. A mutation test was performed for r selected ranges R . Each point was mutated r times, once for each given range. Mutation impact is expressed through the absolute value of the difference between original fitness and fitness after mutation.

3.4.2 Convexity of latent space

Analyzing convexity of latent space is carried out on the same random samples as mutation impact. Representation presents the relation of fitness to mean distance to not worse individuals. Distance is calculated using Euclidean distance in latent space between two given individuals. Visualisation presented this way shows if individuals with better fitness are placed close to each other in latent space.

Chapter 4

Experimental results

This section presents details about conducted experiments. There are multiple ways to convert $f0$ genotypes into graph representation, as well as many available parameters for constructed autoencoder architecture. But because of time limitations, only a selected set of parameters were chosen and tested. Learning experiments were performed on two different data sets, on which two architectural approaches were tested. Graph data used for training was prepared as described in Sect. 3.3. There were multiple preliminary experiments performed to choose what parameters are the most promising. Based on results from preliminary tests, the following parameters were chosen for all models presented in later part of the chapter:

- Batch size – 256,
- n.hidden – 64,
- learning rate – 0.01 with decay,
- optimizer – *Adam optimization*.

4.1 Development of autoencoders

The first set of experiments focused on achieving the lowest reconstruction loss. To limit the number of trained models, the first experiments were conducted only on the velocity data set. To check the impact of different numbers of layers on the training process, only one convolution layer type was used. Restriction to one layer type was necessary because the training process requires over 15 hours for 100 epochs. Time requirements combined with the number of possible options, when considering multiple convolution types, consumes too many resources. That is why the following experiment was conducted on only *GCSconv* convolution layer, which was giving the most promising results in preliminary tests. Experiment compared the following 24 permutations of layers.

- 1,2 or 3 additional convolution layers in encoder,
- 2 additional dense layers in encoder,
- 1 or 2 additional dense layers in decoder A,
- 1 or 2 additional convolution layers in decoder X,
- 2 or 4 additional convolution layers in decoder X.

In the experiment, only *GAE* variant of autoencoder was used with 15 latent space dimensions . For each set of parameters, there were 10 instances of the learning process to achieve reliable results.

Each model was trained for 100 epochs. Fig. 4.2 presents progress of loss for sample run. On that plot, it can be seen that there is a high variation in training reconstruction loss for future matrix X . Oscillations in reconstruction loss A are smaller, compare to reconstruction loss X . Test loss goes down much smoother, compared to training loss. From that plot, it can be seen that longer learning shouldn't bring a big drop in loss, and the biggest gain would most likely be in a lower variation of loss, caused by the decaying learning rate. Fig. 4.1 compares results achieved by given models. Reconstruction loss is the mean value of 10 models trained with the same parameters after 100 epochs. The results have high variation and there is no setting that achieved much better results than the others. The lowest variation is achieved by models with 2 additional convolution layers. Results are further compared to each other using statistic test *nemenyi*, the test showed that the differences are not statistically significant.

The next experiment focused on comparing different convolution types. This experiment tested following parameters:

- different architecture types – *GAE* and *VGAE*,
- different types of convolution layers:
 - *GCSconv*,
 - *GCNconv*,
 - *ARMAconv*,
 - *GATconv*.
- different number of latent space dimensions: 3,10 or 15,
- Additional convolution layers: 1 or 2,
- additional dense layers in encoder – 2,
- additional dense layers in decoder A – 1,
- additional convolution layers in decoder X – 1,
- additional dense layers in decoder X – 2.

Models with all permutations from these settings were trained for 70 epochs. For each set of parameters, 10 runs were performed to collect a bigger sample. Plot of reconstruction loss for *GAE* architecture is visible in Fig. 4.4a, and for *VGAE* in Fig. 4.4b. From these plots, it can be seen that reconstruction loss has a big variance on both architectures. It is also visible that further training shouldn't bring any big gaining in lower loss, as training already slowed. Comparing reconstruction loss A from *VGAE* with reconstruction loss A from *GAE*, it is visible that *VGAE* version has bigger problem with reconstruction matrix A. From all models sharing the same parameters, final losses were averaged and plotted, the results are visible in Fig. 4.3. The comparison shows that there is a big difference between latent space of size 3 and the rest. Models with latent space of size 3 achieved average reconstruction loss worse by 200 to 400 compared to their counterparts with latent space of size 10 and 15. The difference between models with latent space of size 10 and 15 is not statistically significant. Comparing *GAE* to its variational counterpart, it can be seen that models using *GAE* achieve slightly lower reconstruction loss. The reason behind that is

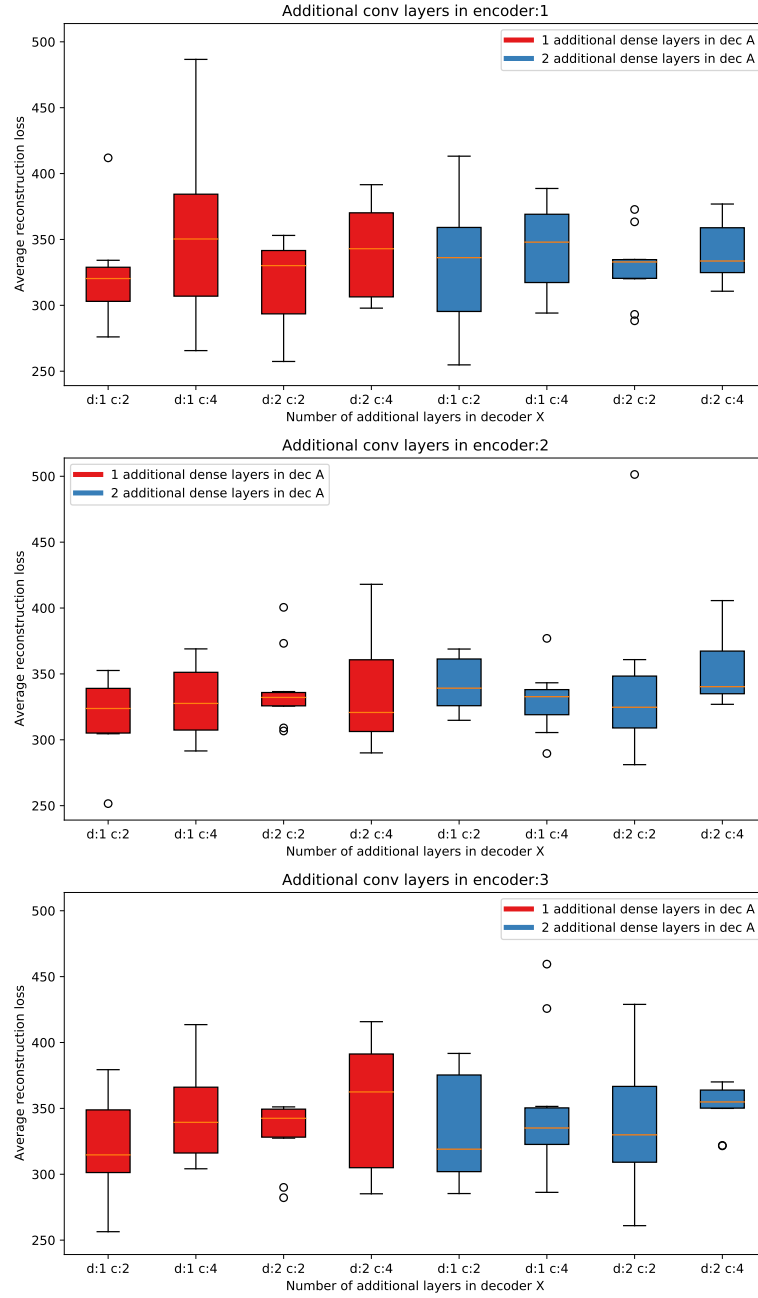


Figure 4.1: Comparison of reconstruction loss for different settings of layers. Reconstruction loss is taken from 10 separate training runs, with the same parameters.

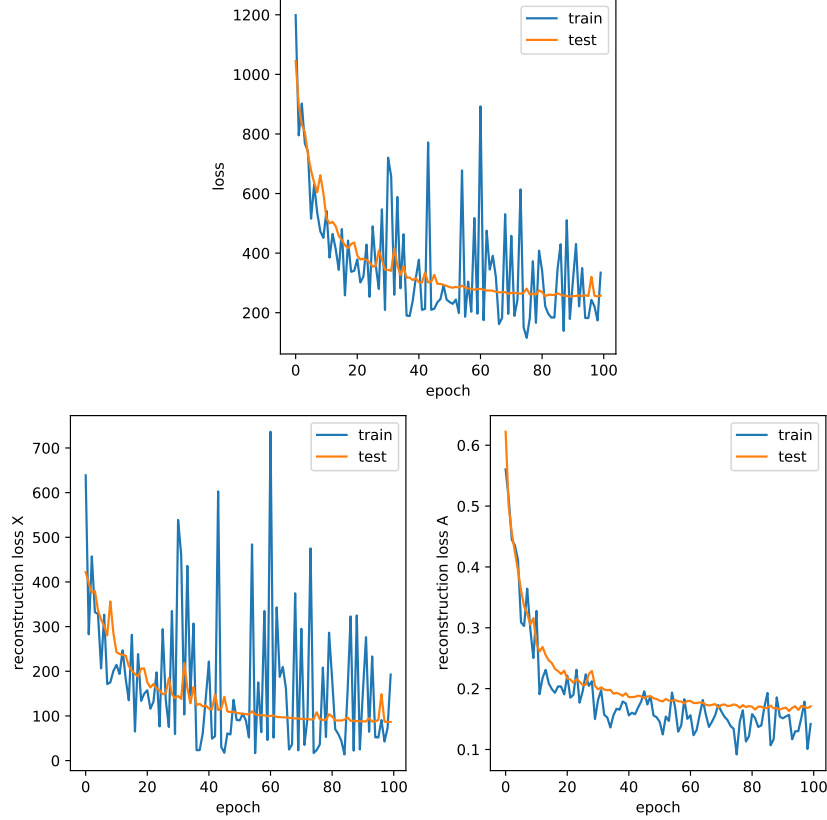


Figure 4.2: Learning process of GAE architecture with GCSConv layers. Impact of reconstruction loss A on the model is 1000 times bigger than what is seen on the figure, because of used weight.

previously mentioned reconstruction loss A, which is consistently higher for *VGAE* architecture. Across subplots in Fig. 4.3 presenting latent space of size 10 and 15, *Armaconv* and *GCSConv* obtain slightly lower average loss.

Both experiments didn't bring the definitive best set of parameters, as the results have high variance and the difference between them is not enough. But because of the need to reduce the number of tested parameters for the next experiments, *GCSConv* was selected as the base convolution layer type.

4.1.1 Latent space properties

The following experiment focused on testing latent space properties of trained models. From each set of parameters, a model with the lowest final reconstruction loss was chosen. Each best model was sampled and analyzed using methods described in Section 3.4. Fig. 4.5a and Fig. 4.5b present impact of mutation for *GAE* architecture. Results differ from each other based on whether examples are sampled from latent space or data set. In the case of examples sampled from latent space, mutation on average is weaker compare to examples drawn from the data set. The reason for this comes from the irregular distribution of latent space. Searching through latent space in some areas can give more variety than in others as auto encoder does not use the whole spectrum of latent space. Because of this samples from the data set are condensed closer to each other in latent space. Proof of that is visible when comparing Fig. 4.6a and Fig. 4.6b. On these figures it can be seen that random samples from the data set are much closer to each other compare to random samples from latent space.

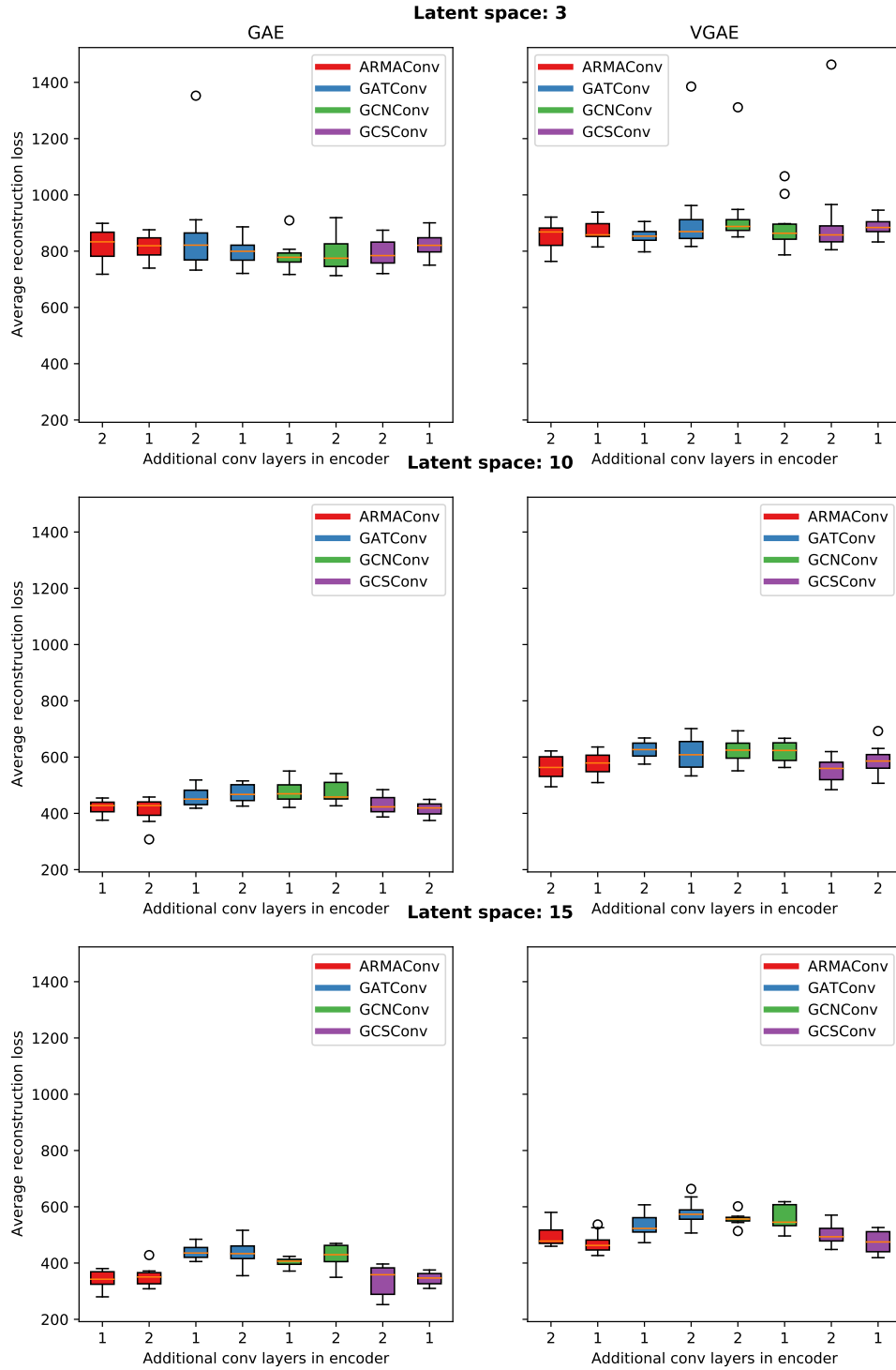


Figure 4.3: Average final reconstruction loss of 10 models for each set of parameters.

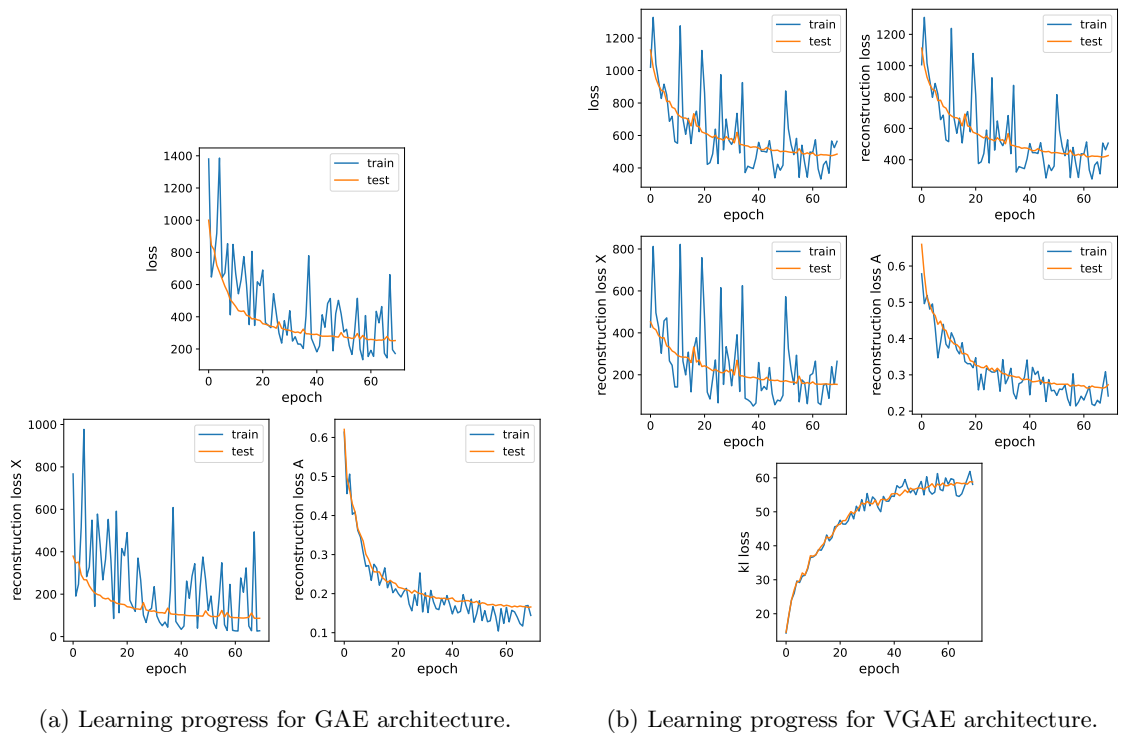
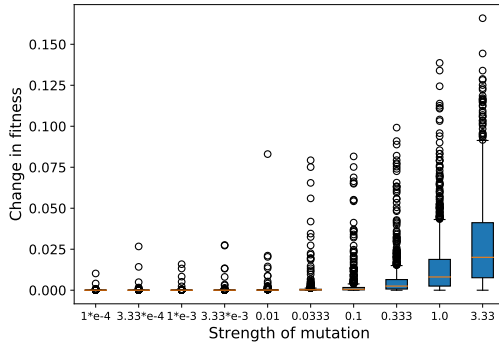
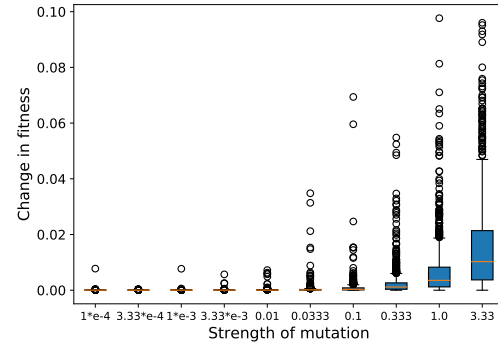


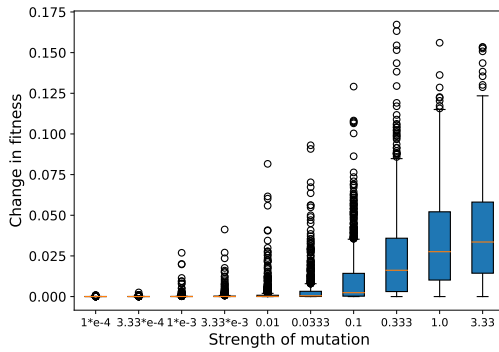
Figure 4.4: Comparison of learning progress for base GAE and VGAE architectures trained on velocity data set.



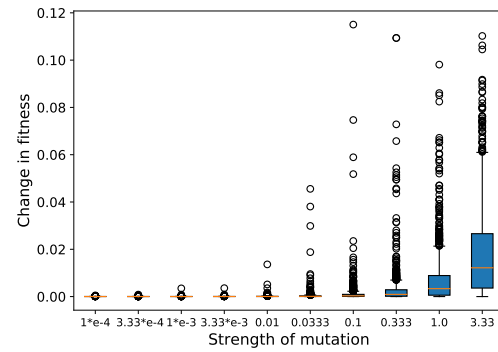
(a) Mutation impact on random samples from latent space of GEA architecture.



(c) Mutation impact on random samples from latent space of VGEA architecture.



(b) Mutation impact on random samples from velocity data set of GEA architecture.



(d) Mutation impact on random samples from velocity data set of VGEA architecture.

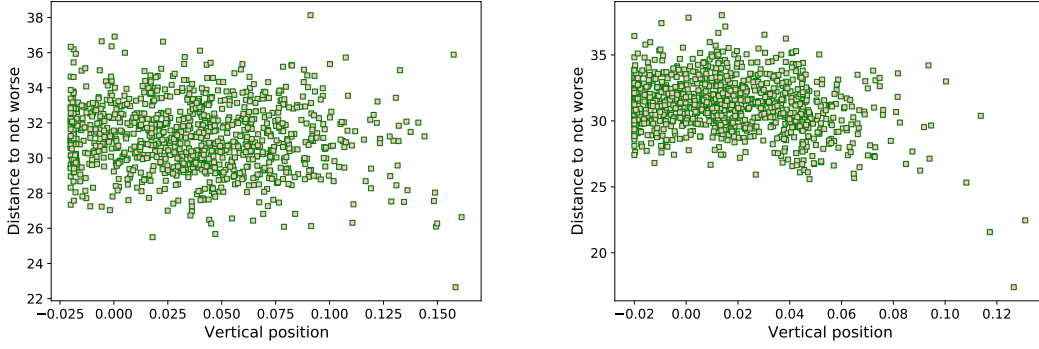
Figure 4.5: Mutation impact for GAE and VGAE architecture with latent space 15 and no locality loss.

Comparing Fig. 4.5a and Fig. 4.5b with Fig. 4.5c and Fig. 4.5d shows that mutation in *VGA* is different compare to its *GAE* counterpart. The difference in mutation strength across examples from data set and examples from latent space is not as big as in *GAE* model. The variational model spreads individuals more across latent space which is visible in Fig. 4.6d. While all sampled individuals from *GAE* were on average between 1 to 6 distance to not worse individuals, in *VGAE* architecture this distance is between 7 to 18. Figures for other parameters were not shown as the results presented by them does not differ much from what was presented.

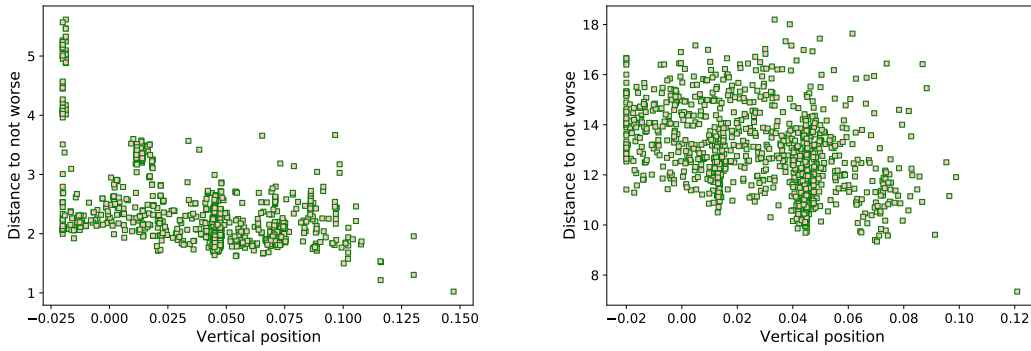
4.2 Development locality loss

The next experiments focused on the testing impact of developed locality loss. Because of the long training session number of tests that could be performed was limited. Based on results from previous experiments set of parameters was chosen. All experiments were conducted for both *GAE* and *VGAE* architecture. Both architectures had the same parameters which are:

- latent space – 15,
- additional convolution layers in encoder – 2,
- additional dense layers in encoder – 2,
- additional dense layers in decoder A – 1,



(a) Dist to not worse individuals on random samples from latent space of GEA architecture. (c) Dist to not worse individuals on random samples from latent space of VGAE architecture.



(b) Dist to not worse individuals on random samples from velocity data set of GEA architecture. (d) Dist to not worse individuals on random samples from velocity data set of VGAE architecture.

Figure 4.6: Dist to not worse individuals for GAE and VGAE architecture with latent space 15 and no locality loss.

- additional convolution layers in decoder $X - 1$,
- additional dense layers in decoder $X - 2$,
- convolution layer type – *GCSconv*.

For each set of parameters, 10 models were trained for 100 epochs.

4.2.1 Locality based on the number of parts

Learning progress from Fig. 4.7 shows that *GAE* variant struggles with learning custom loss, while *VGAE* version lowers it over the epochs. Part loss in Fig. 4.7a rises to over 1100, which means that there is a negative correlation of around 0.1 between the average distance between individuals and their number of parts. In the case of *VGAE*, part loss stabilizes around 500, which means a correlation of 0.5 between the number of parts that individuals have and the distance between them. Comparison of Fig. 4.8a with Fig. 4.8b shows that mutation operator in *GAE* variant has bigger impact on ranges from 0.0333 to 3.33 compare to *VGAE* variant. Fig. 4.8c and 4.8d shows a small tendency between bigger vertical position and smaller distance to not worse individuals, which implies that better individuals are slightly closer to each other. It is especially visible with outliers that have the highest vertpos and are on average the closest to not worse individuals. Analyze of difference between that figure and an example of the same architecture without locality

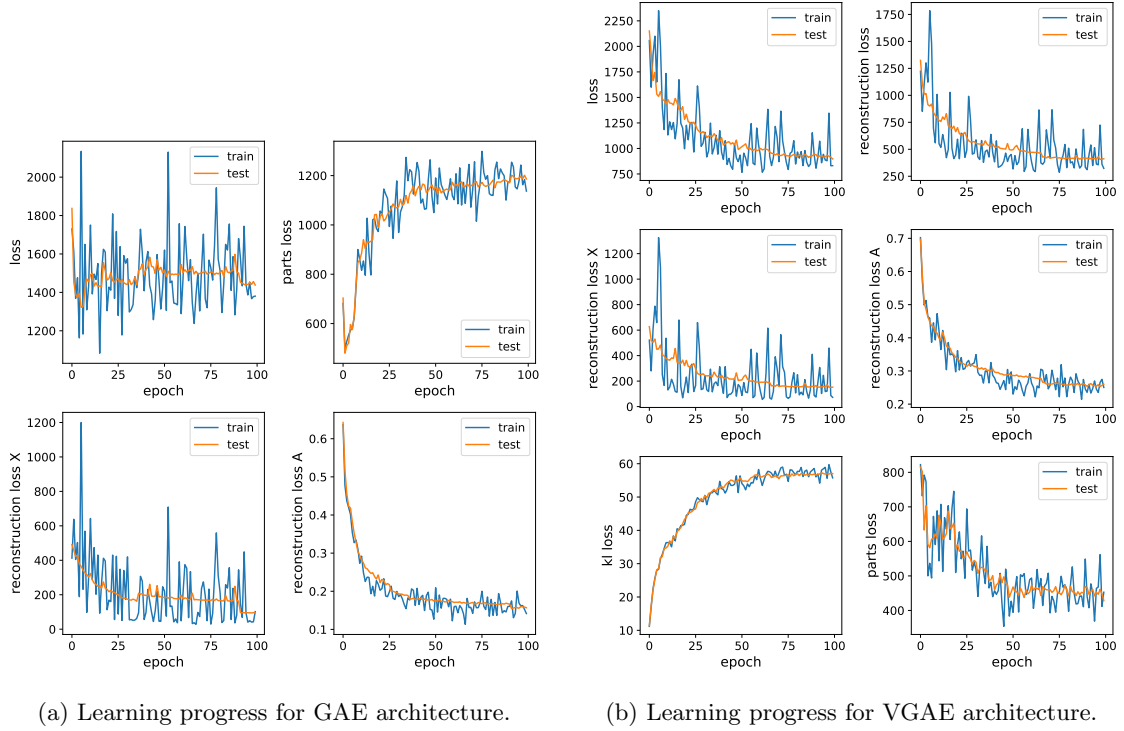


Figure 4.7: Learning progress for models with locality loss encouraging similar number of parts.

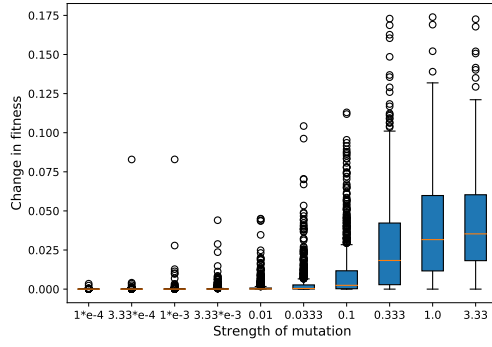
loss presented in Fig. 4.6b, shows that the cloud of points from the model that uses locality loss, leans slightly more to the closer distance between better individuals. However, the difference is not big enough to make a definitive statement, as well as the difference, can be caused by the difference in sampled points.

4.2.2 Locality based on similarity measure

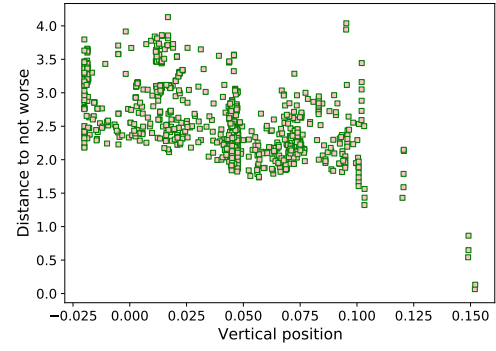
Looking at Fig. 4.9a it is visible on learning progress that *GAE* architecture managed to get a positive correlation of 0.3 between similarity of individuals and their distance in latent space. *VGAE* architecture performed better at learning custom loss, and managed to achieve a positive correlation of around 0.4. The introduction of locality loss didn't affect reconstruction losses, as a variation of reconstruction loss is still high. Comparing plots in Fig. 4.10 it is visible that also for this locality loss, mutation performed on *GAE* architecture gives a much bigger impact than the same mutation on *VGAE*. There is also slight increase in number of outliers for lower mutation strength in *VGAE* architecture, compare to version that used part locality loss or no locality loss at all. But difference is too small to call it significant. Fig. 4.10d shows slightly bigger trend in smaller distance to not worse individuals, which can be the impact of similarity locality loss, as it has direct translation in to vertpos value.

4.2.3 Locality based on fitness

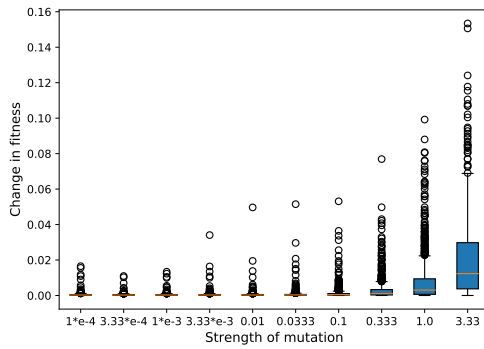
Compare to other custom losses, fitness locality loss proved to be the hardest to learn, especially for *GAE* variant. Fig. 4.11a shows that *GAE* version was oscillation on correlation level between -0.1 and 0.05 which means that it was unable to learn locality correctly. *VGAE* variant showed a bit better results and achieved a positive correlation of around 0.25 , which is visible in Fig. 4.11b. Compared to other locality losses, fitness-distance correlation turned out to be the weakest on



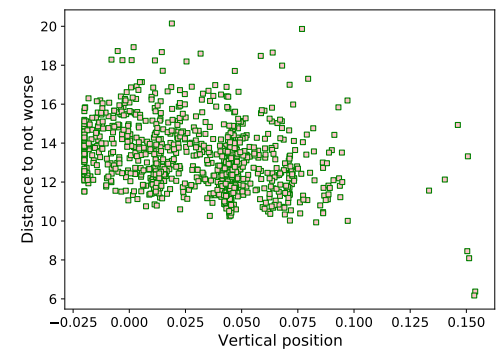
(a) Mutation impact on random samples from velocity data set for GAE architecture.



(c) Average distance to not worse individuals for GAE on random samples from velocity data set.

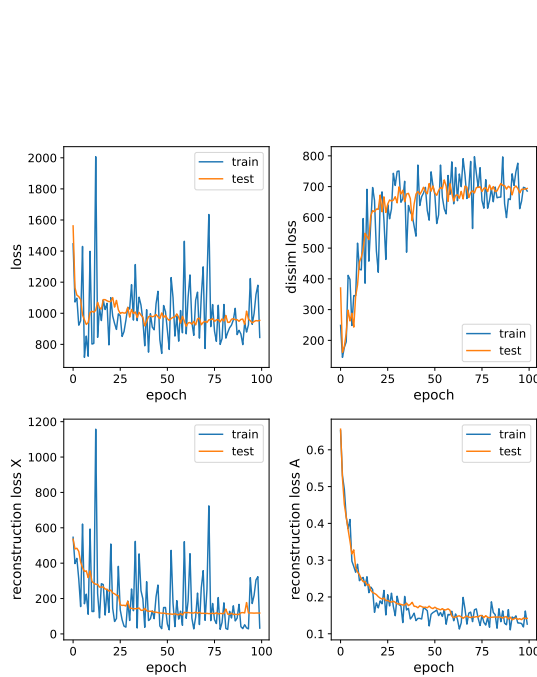


(b) Mutation impact on random samples from velocity data set for VGAE architecture.

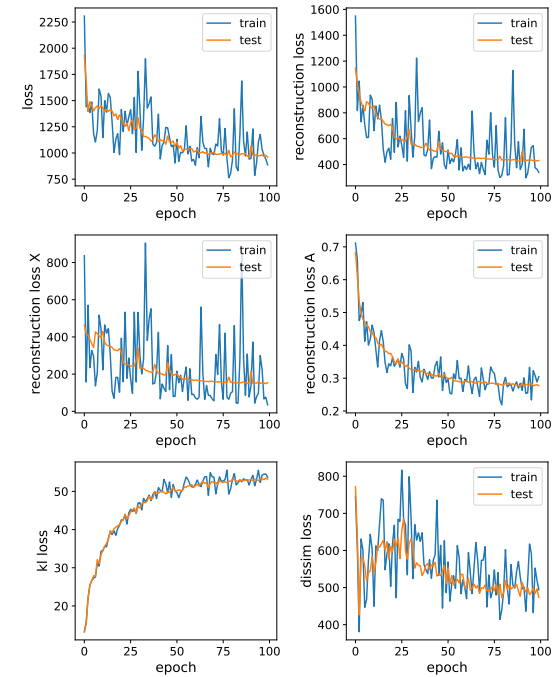


(d) Average distance to not worse individuals for VGAE on random samples from velocity data set.

Figure 4.8: Mutation impact and distance to not worse for VGAE and GAE architectures with latent space of size 15 and locality loss encouraging similar number of parts.

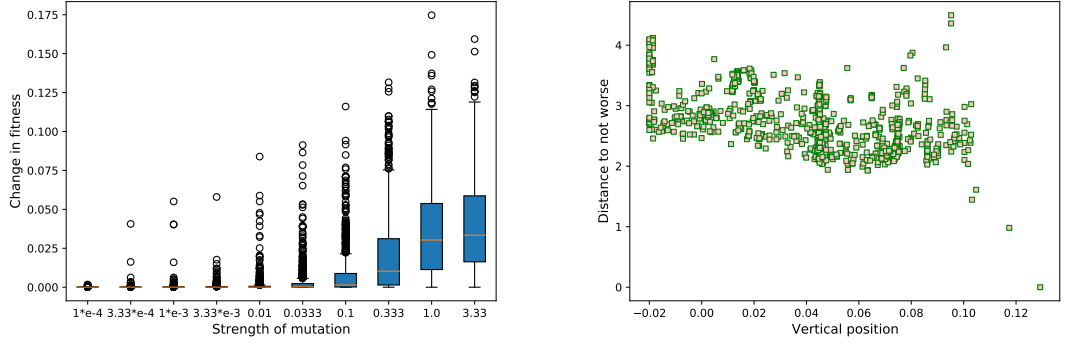


(a) Learning progress for GAE architecture.

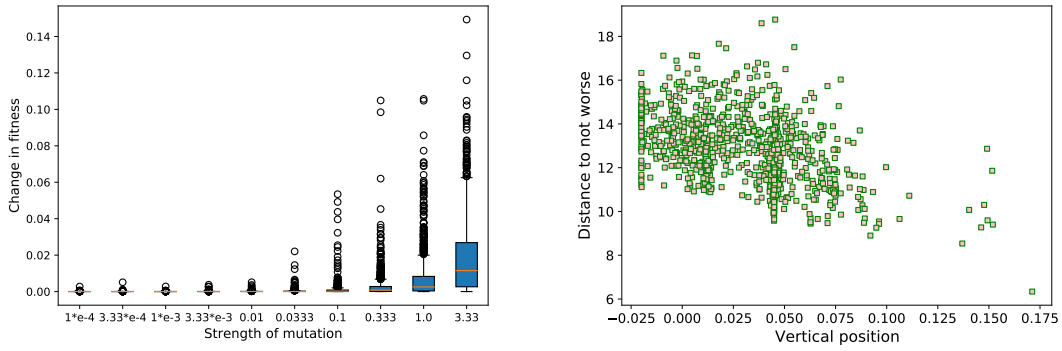


(b) Learning progress for VGAE architecture.

Figure 4.9: Learning progress for models using custom similarity loss based on graph similarity.



(a) Mutation impact on random samples from velocity data set for GAE architecture. (c) Average distance to not worse individuals for GAE on random samples from velocity data set.



(b) Mutation impact on random samples from velocity data set for VGAE architecture. (d) Average distance to not worse individuals for VGAE on random samples from velocity data set.

Figure 4.10: Mutation impact and distance to not worse for VGAE and GAE architectures with latent space of size 15 and loss encouraging locality for similar individuals using graph similarity.

both architectures. The reason for this can come from the nature of locality loss, as determining the correlation between the genotype and fitness is the hardest compared to the three locality types used. Fig. 4.12a and Fig. 4.12b show, that mutation impact of *GAE* is still bigger than in variational variant. Fig. 4.12c shows tendency that individuals with better fitness are closer to each other. There are also a few individuals which confirm that as they have highest vertpos sampled and are located very close to each other. There are no significant changes in mutation strength with fitness locality loss. Which is to be expected as correlation between distance and fitness was weak, so the impact of locality loss should be also weak.

4.2.4 Summary of locality losses

Conducted experiments showed, that *GAE* architecture struggles more with learning locality loss. *VGAE* variant was able to achieve better correlations, compare to *GAE*. Both architectures were not able to achieve correlation over 0.5 on given data set for any given locality type. For the velocity data set, fitness locality loss turned out to be the hardest to learn. Both architectures were unable to achieve a good correlation between the fitness and distance of individuals in latent space. For a given data set, locality losses didn't have high mutation strength for tested ranges of mutation. The same can be mentioned about the impact on the distance to not worse individuals. There could be seen a correlation between higher fitness and closer distance to not worse individuals, but there was no definitive impact produced by locality loss.

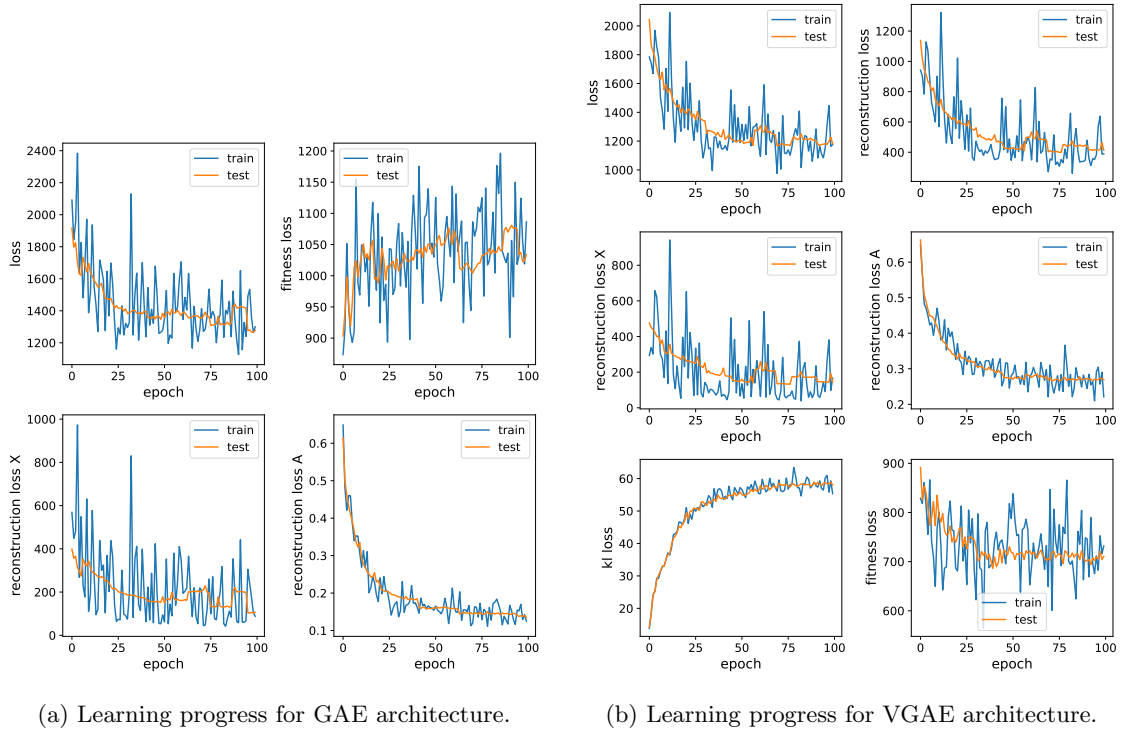


Figure 4.11: Learning progress for loss encouraging fitness locality.

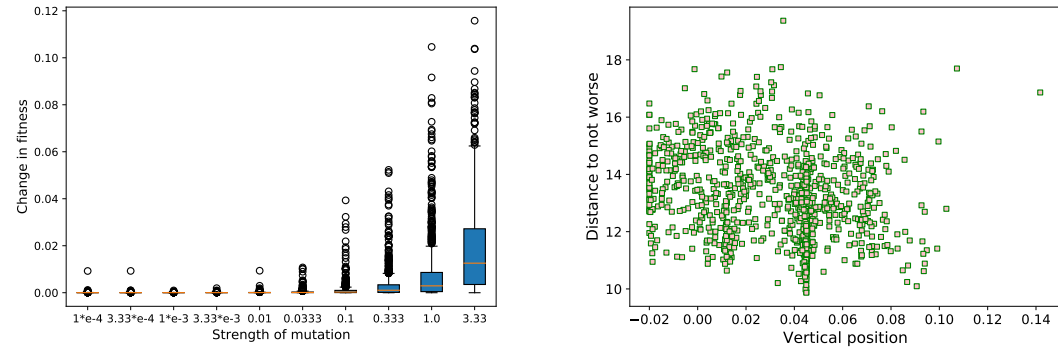
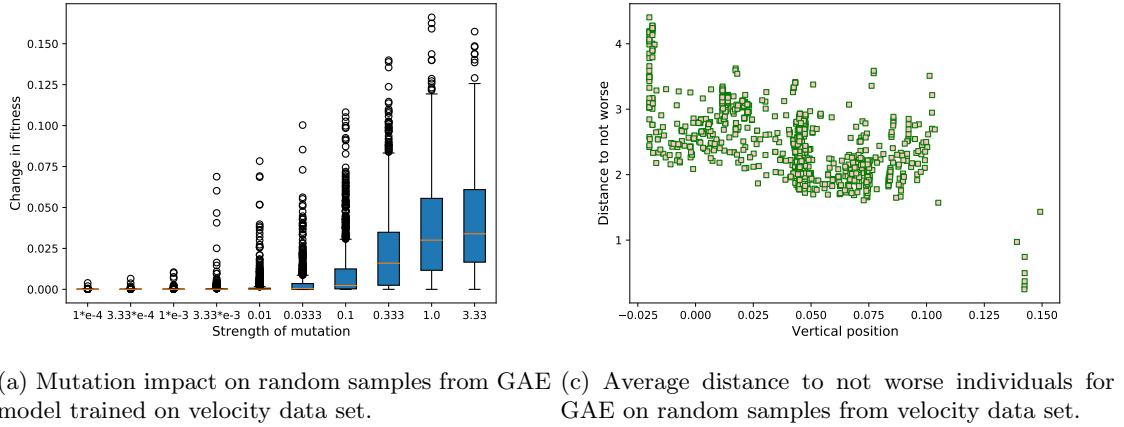


Figure 4.12: Mutation impact and distance to not worse for VGAE and GAE architectures with latent space of size 15 and loss encouraging locality for individuals with similar fitness.

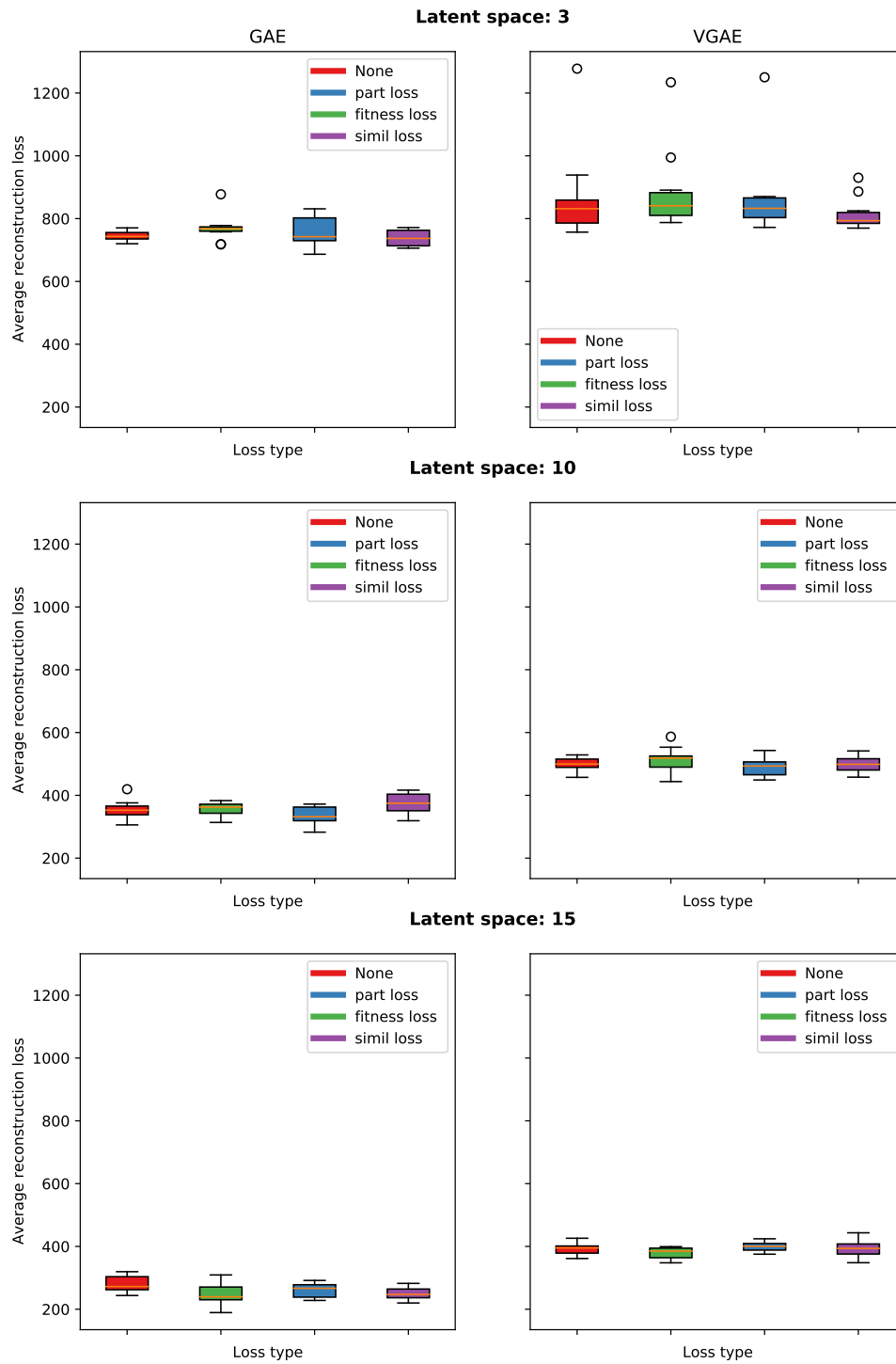


Figure 4.13: Comparison of reconstruction losses for models trained on vertpos data set. Each bar represents 10 training runs performed with given setting. Number next to latent space refers to number of dimensions.

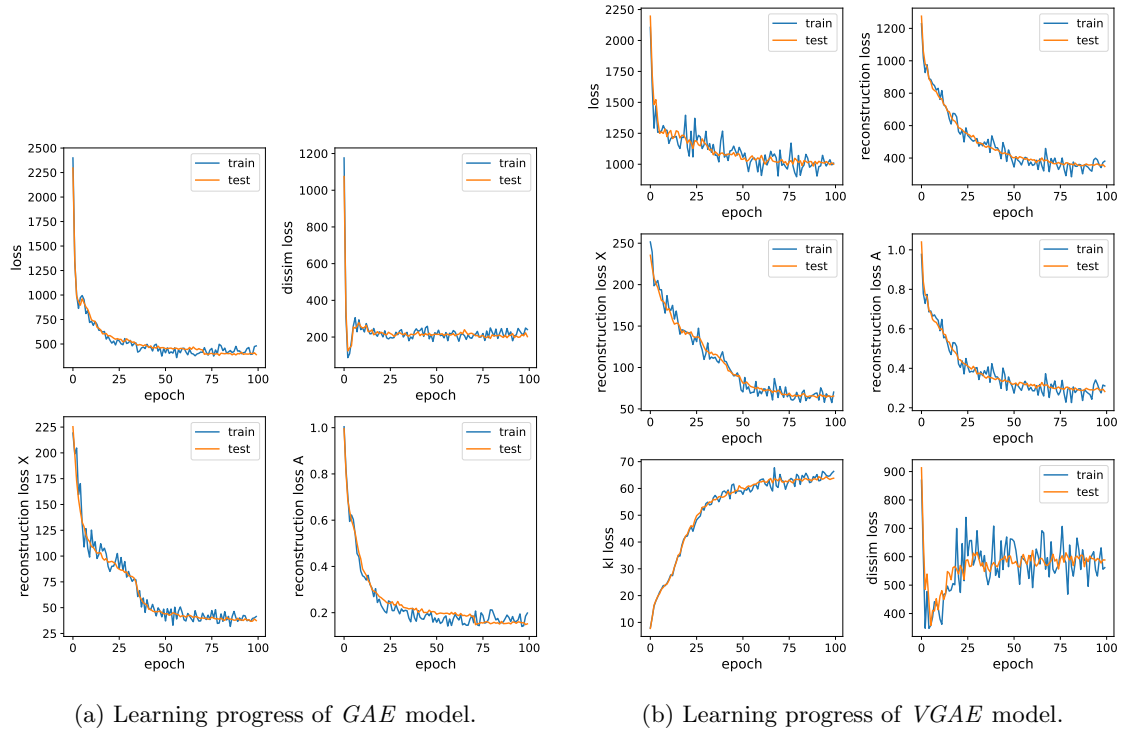


Figure 4.14: Learning progress for *GAE* and *VGAE* architecture with similarity locality loss. Trained on vertpos data set.

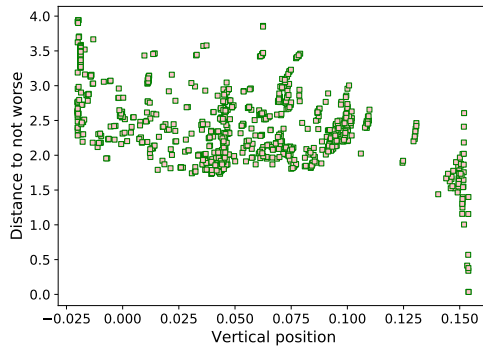
4.3 Vertpos data set

The next part focuses on experiments that were performed on *vertpos data set*. The same as with previous experiments, due to time limitations number of parameters tested on this data set had to be reduced. Based on results from previous experiments following parameters were chosen:

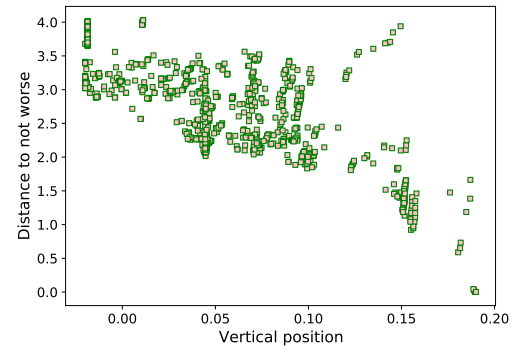
- Architecture type – *GAE* and *VGAE*,
- Latent space size – 3,10 and 15,
- additional convolution layers in encoder – 2,
- additional dense layers in encoder – 2,
- additional dense layers in decoder A – 1,
- additional convolution layers in decoder X – 1,
- additional dense layers in decoder X – 2,
- convolution layer type – *GCSconv*,
- loss types:
 - None – only base reconstruction loss,
 - Part locality loss,
 - Similarity locality loss,
 - Fitness locality loss.

All sets of parameters were trained 10 times each. Comparison of reconstruction losses is presented in Fig. 4.13. From the comparison it can be seen, that as previously models struggle to learn representation on latent space with numbers of dimension equal to 3. Latent space of with 15 dimensions produces models with reconstruction loss lower by 100 on average, compare to latent space with 10 dimensions. There is also small difference in reconstruction losses between *GAE* and *VGAE* variation. The difference comes mostly from the reconstruction of A with which *VGAE* struggles more than *GAE*. From that figure it can be also seen, that additional locality loss, does not have any significant impact on final reconstruction loss.

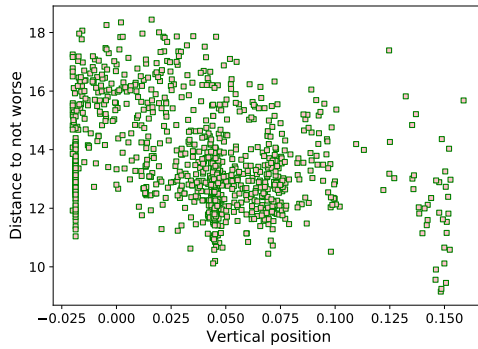
Learning conducted on verpos data set resulted in more stable learning progress which can be both seen on comparison plot (Fig. 4.13) and learning progress plot (Fig. 4.14a). Comparing locality losses from this experiment and the previous one, the biggest difference is visible on *GAE* architecture using similarity locality loss. Learning progress demonstrating it is visible in Fig. 4.14a. Models trained on verpos data set achieved much better distance–similarity correlation compare to models with the same parameters trained on velocity data set. The reason for this can come from data set specifications. Verpos data set differs from velocity data set in that it has more diverse shapes of individuals. Genotypes evolved for greater velocity, characterized by smaller bodies with a lower number of parts, compared to individuals with various branches which are results for evolving aiming at the highest vertical position. Results in mutation strength across all models trained on verpos data set do not have any significant differences compared to their equivalents from velocity data set. Mutation impact on *GAE* architectures is higher compare to *VGAE* variation. Fig. 4.15c shows the impact of similarity locality loss which achieves has high correlation value. Points representing individuals with higher verpos values are placed closer to better individuals than the ones with lower verpos values. It is worth mentioning that fitness locality loss performed worse compared to similarity local loss, even though he impacts fitness–distance correlation directly. Similarity locality loss performs well in that scenario because of the connection between vertical positioning and similarity of individuals, as high individuals are often similar to other high individuals.



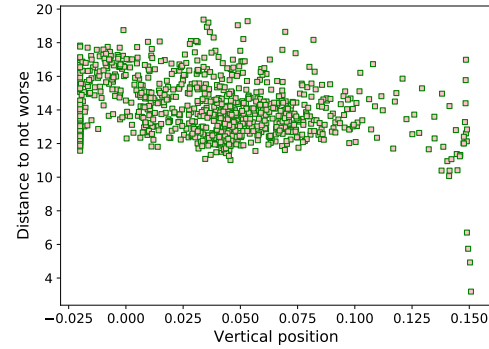
(a) Average distance to not worse individuals for GAE on random samples from vertpos data set with fitness locality loss.



(c) Average distance to not worse individuals for GAE on random samples from vertpos data set with similarity locality loss.

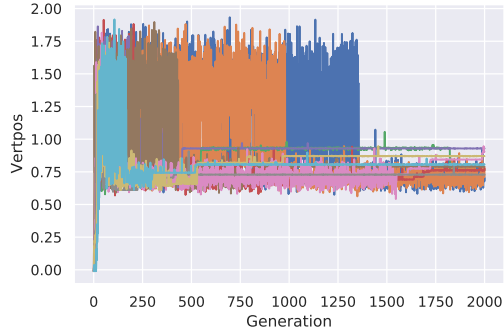


(b) Average distance to not worse individuals for VGAE on random samples from vertpos data set fitness locality loss.

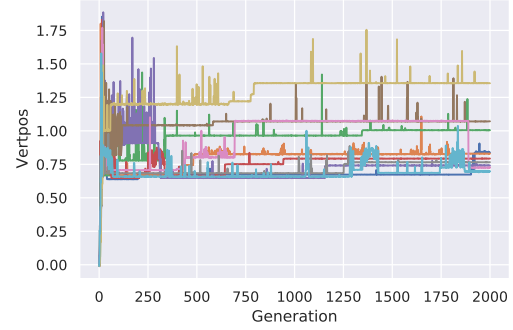


(d) Average distance to not worse individuals for VGAE on random samples from vertpos data set similarity locality loss.

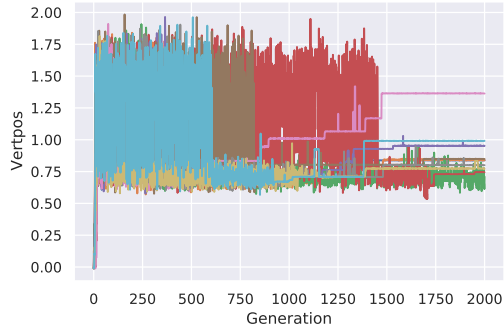
Figure 4.15: Comparison of average distance to not worse individuals for models learned on vertpos data set.



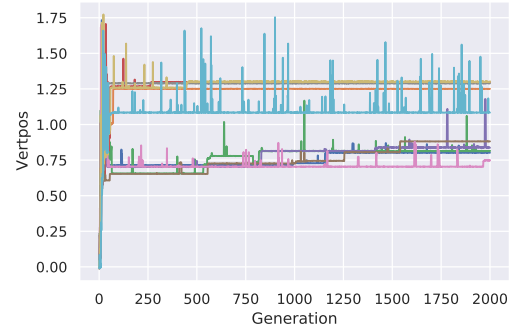
(a) Evolutionary runs using mutation of strength 0.0333 performed on *VGAE*. Model trained using part locality loss.



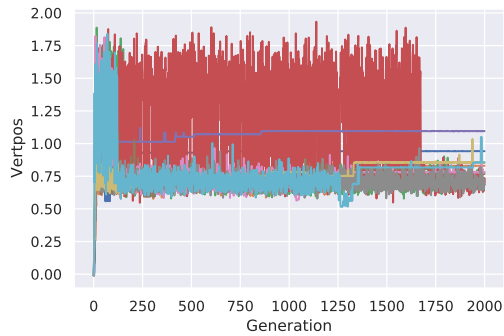
(d) Evolutionary runs using mutation of strength 0.0333 performed on *VGAE*. Model trained using similarity locality loss.



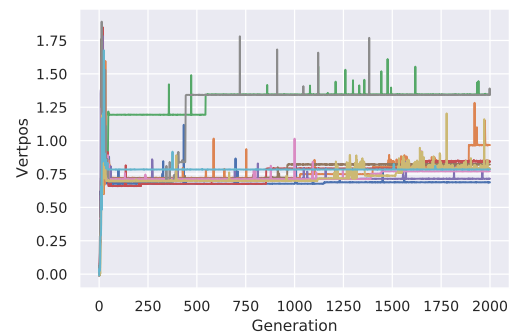
(b) Evolutionary runs using mutation of strength 0.01 performed on *VGAE*. Model trained using part locality loss.



(e) Evolutionary runs using mutation of strength 0.1 performed on *VGAE*. Model trained using similarity locality loss.

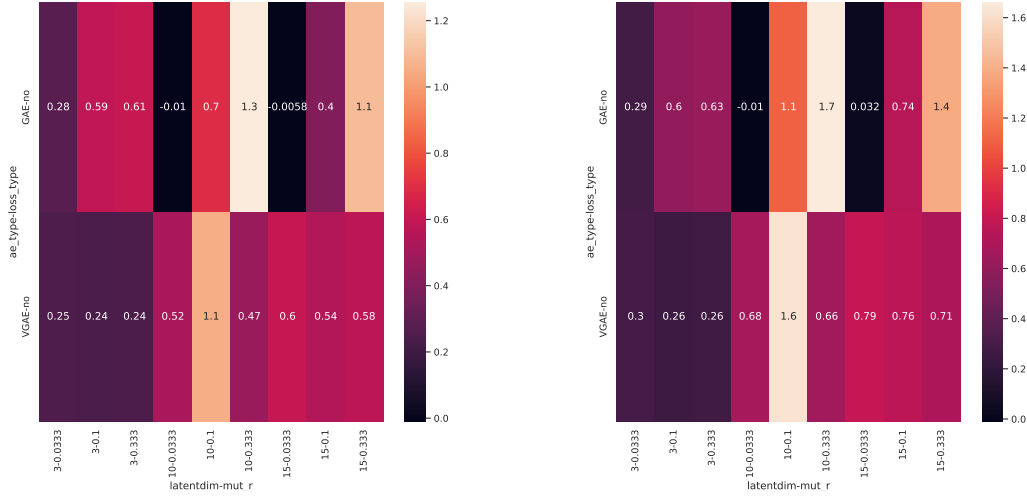


(c) Evolutionary runs using mutation of strength 0.333 performed on *VGAE*. Model trained using part locality loss.



(f) Evolutionary runs using mutation of strength 0.333 performed on *VGAE*. Model trained using similarity locality loss.

Figure 4.16: Comparison of evolution progress for two *VGAE* models trained using vertpos data set with two different locality losses. Each plot contains 10 runs sharing the same setups.



(a) Average verpos achieved across 10 evolution runs, performed on model for given settings. (b) Max verpos achieved across 10 evolution runs, performed on model for given settings.

Figure 4.17: Results of evolution conducted on best models trained on velocity data set with no locality loss.

4.4 Evolutionary experiments

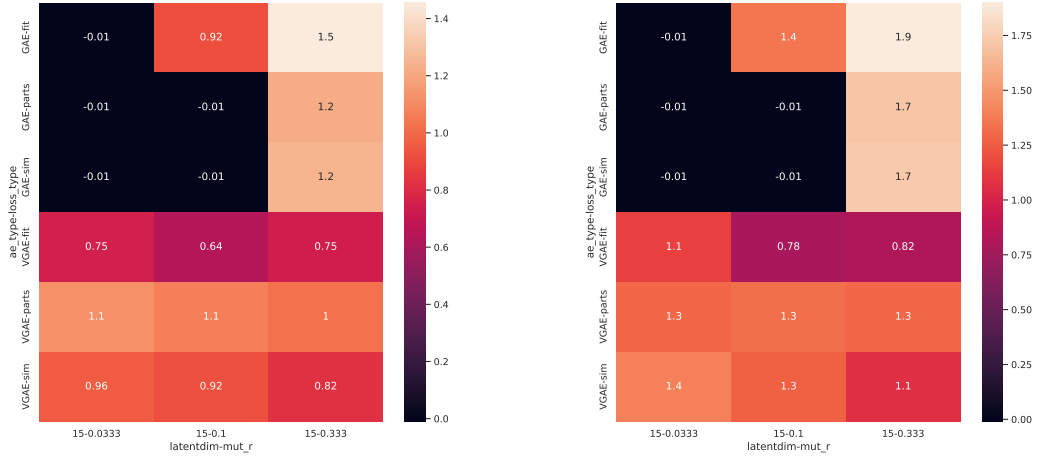
Evolutionary experiments were carried out according to description from Sect. 3.3.3. Performed experiments can be divided into 3 sections based on what models were used for training:

- models trained on velocity data set without locality loss,
- models trained on velocity data set with locality loss,
- models trained on verpos dat set.

For each set of experiments, the best models were selected. Selection was based on reconstruction loss achieved on training set for both models with locality loss and with out it. The reason for not accounting locality loss during selection is that the differences between models trained with the same parameters were mainly visible in reconstruction loss. Another reason for that choice is to minimize the error coming from wrong reconstruction.

Looking at the Fig. 4.16, it can be seen, that locality losses produce different latent spaces that have different impact on evolution. Both models presented in Fig. 4.16 were trained using the same parameters with the difference being locality loss. Model trained on Part locality loss shows big jumps in fitness across all mutation ranges. Evolutionary run using similarity locality loss presents less jumps. Especially for mutation range 0.0333 visible in Fig. 4.16d there are runs which steadily improved max fitness.

Results of evolution on models trained on velocity data set with out locality loss are shown in Fig. 4.17. From the heat map, it can be seen that latent space of size 3 on average performed the worst, compare to models trained on bigger latent spaces. Another finding is that *GAE* version struggles to perform evolution, when mutation strength is small. the reason for this may be that evolution starts with very simple individuals, after translating those individuals in to latent space, their distance to valuable individuals is too far, and because of this evolution struggles with reaching better results. Using stronger mutation helps, as there is higher chance that evolution will be able to travel that distance in latent space. *VGAE* variant looks resistant to it, as the



(a) Average vertpos achieved across 10 evolution runs performed on model for given settings. (b) Max vertpos achieved across 10 evolution runs performed on model for given settings.

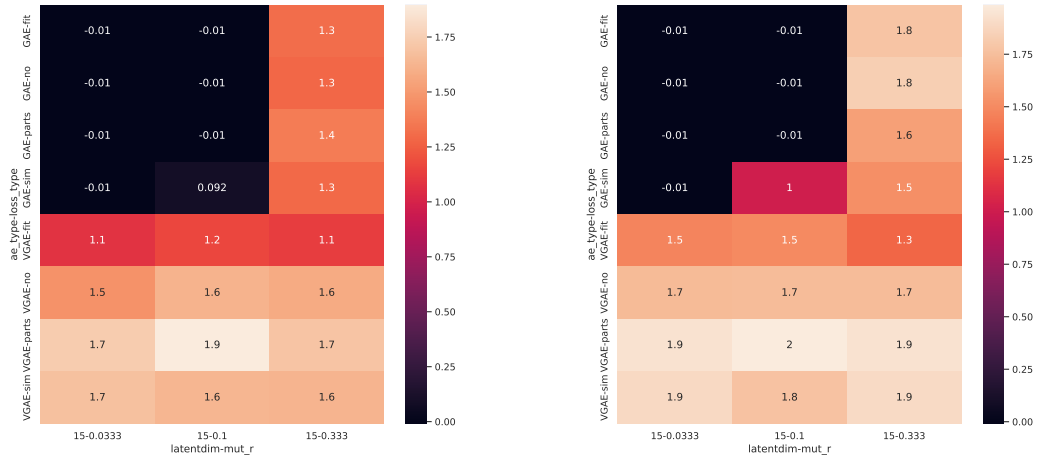
Figure 4.18: Results of evolution conducted on best models trained on velocity data set with locality loss.

distribution in the latent space is more scattered. Fig. 4.17b shows that the best result archived evolution performed on model using latent space of size 10.

Results of evolution using models trained with locality loss are shown in Fig. 4.18. Usage of parts and similarity locality loss, further increase problem of *GAE* of starting evolution process. Comparing Fig. 4.17a with Fig. 4.18a shows that usage of locality loss improved average vertpos achieved during evolution. The same relationship is visible with maximum vertpos achieved which is visible by comparing Fig. 4.17b with Fig. 4.18b. Is it also visible that models trained on velocity data set with *GAE* architecture, outperform *VGAE* variant as long as they begin evolution correctly.

Last evolutionary runs were performed using models trained on vertpos dat set. Comparison of results is visible in Fig. 4.19. *GAE* models trained on vertpos data set, show the same problem as models trained on velocity data set. It can also be seen, that *VGAE* architecture performs better than *GAE* architecture on vertpos data set.

Fig. 4.20 compares best individuals from both data sets, with baseline evolution. As baseline, 10 evolutionary runs using Framsticks *f0* representation were used. The summary comparison shows that evolution carried out on models trained on vertpos data sets outperforms evolution carried out on models trained on velocity data sets. It can be also seen, that which architecture type performs better, depends heavily on training data set. On velocity data set, *GAE* performs better, while on vertpos data sets *VGAE* achieves better results. Comparing results to baseline, it can be seen that models trained on vertpos data sets in their best evolution, achieved a higher median over baseline. Unfortunately, even the best evolutions conducted on latent space were not able to achieve higher maximum vertpos than baseline.



(a) Average vertpos achieved across 10 evolution runs performed on model for given settings. (b) Max vertpos achieved across 10 evolution runs performed on model for given settings.

Figure 4.19: Results of evolution conducted on best models trained on vertpos data set with locality loss.

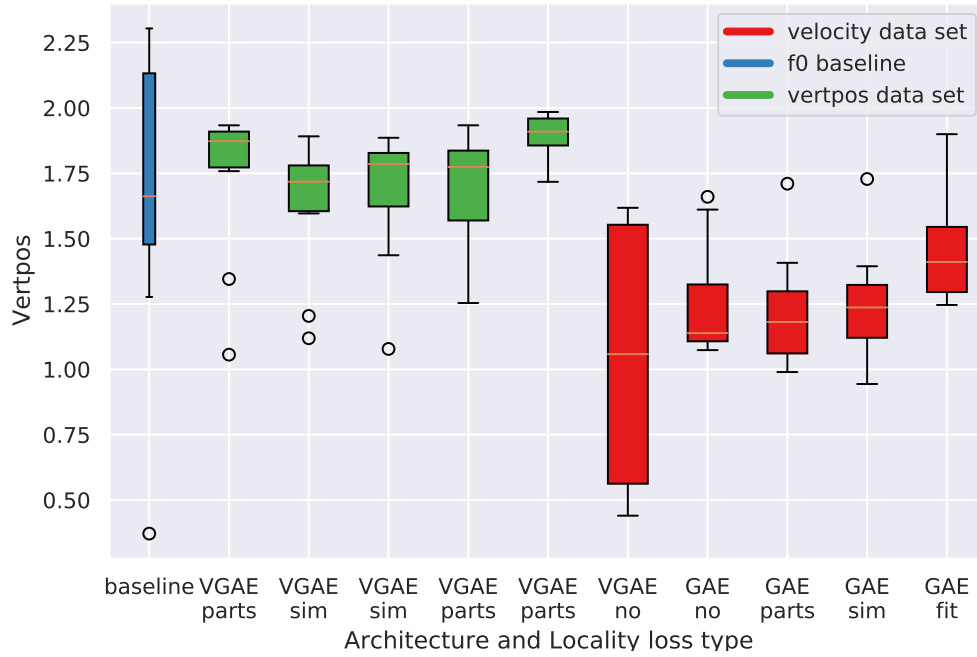


Figure 4.20: Comparison of 5 best models for each data set with baseline. Each box presents 10 runs.

Chapter 5

Conclusions

This chapter provides summary of what was achieved, key finding and problems, as well as presents suggestions for future work.

5.1 Summary

All objectives planned for this work were completed successfully. The goal of this thesis was to build and test autoencoder-based architecture, which will be suitable for creating a new representation, that will be used during optimization processes. For this purpose, a graph-based autoencoder was created. Architecture offers multiple different settings enabling selection between base variant (*GAE*) and variational variant (*VGAE*), as well as multiple layers configurations which include the number of layers in each component, and convolution layer type that is used for graph operations. For the purpose of landscape smoothing, three additional loss types were suggested and implemented. These losses focus on aspects such as, locality of the number of parts in the 3D structure, locality of fitness, and phenotype similarity locality which encourages encoding similar 3D structures close to each other in the latent space. The proposed architecture uses components based on Keras [8] and Tensorflow 2 API [2]. Multiple experiments aiming at testing the training capabilities of architecture were conducted. Experiments aimed at testing: re-constructual capabilities, the impact of mutation in latent space, the convexity of latent space as well as evolutionary capabilities of trained models.

5.2 Key points

Performed experiments indicate that the proposed approach can successfully create new representations. These representations can be further used for evolutionary optimization. Created architecture in combination with *EA* managed to produce individuals with better fitness values than what was used to train the models. Training experiments showed that *VGAE* architecture struggles more with the reconstruction of adjacency matrix, compare to *GAE* variant. *VGAE* variant spreads individuals more, which causes mutation on latent space to have a more even impact. *GAE* variant arranges individuals from data set closer to each other, which produces a more uneven landscape. Because of this, changes in the impact of the mutation differ, based on searching space. *GAE* variant also struggles with the beginning of evolution. Because of the uneven spread of solutions across latent space, it is harder to begin evolution. When locations of starting individuals are far from valuable space and mutation impact is low, *EA* struggles to navigate in latent space. Fitness locality loss proved to be the hardest to learn in every conducted experiment that used it.

5.3 Future work

There are many possible ideas for extensions or changes to architecture that emerged during the experimental phase. The first idea targets the representation of non-existence parts, as prohibiting negative values, can have an unwanted impact on the training process. Graph representation can be also extended by more features. Possible new features are, more details about parts and the addition of neurons to enable active evolution. Based on the differences between the two data set, it is visible that data with greater variety of genotypes, has a positive effect on both learning and future use of latent representation for *EA*. That is why there is a suggestion to implement additional data augmentation, which creates more individuals for training based on simple mutations. Evolutionary experiments showed a tendency to get stuck in the areas of latent space, that are not producing valuable results. There are two suggestions for solving mentioned problem. The first suggestion relies on the adaptive mutation force. During the evolution process, if evolution can not leave the neighbourhood and achieve valuable results, mutation force is increased until non zero results are in the population. The weakness of this solution is that there is no guarantee of success, but as it was seen during evolutionary experiments, runs with higher mutation force, tend to find valid samples more often. A second suggestion is based on analyzing how model scattered individuals in latent space, and based on that observation generate random samples from limited space. Because of the limitation of autoencoders in generative capabilities, the idea for performing evolution and training simultaneously arose. The *EA* algorithm would be able to generate new samples that autoencoder never learned, while the autoencoder could smoothen landscape and help find correct direction for *EA*.

Bibliography

- [1] The evolalg python module for evolutionary algorithms, 2021.
<https://www.framsticks.com/svn/framsticks/framspy/evolalg/>.
- [2] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org.
- [3] Rajesh Kumar Arora. *OPTIMIZATION, Algorithms and Applications*. Chapman & Hall/CRC, 2015.
- [4] Filippo Maria Bianchi, Daniele Grattarola, Lorenzo Livi, and Cesare Alippi. Graph neural networks with convolutional arma filters. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, page 1–1, 2021.
- [5] Michael M Bronstein, Joan Bruna, Yann LeCun, Arthur Szlam, and Pierre Vandergheynst. Geometric deep learning: going beyond euclidean data. *IEEE Signal Processing Magazine*, 34(4):18–42, 2017.
- [6] Thomas Bäck, Günter Rudolph, and Hans paul Schwefel. *Evolutionary Programming and Evolution Strategies: Similarities and Differences*, pages 11–22.
- [7] Kyunghyun Cho. Boltzmann machines and denoising autoencoders for image denoising, 2013.
- [8] François Chollet et al. Keras. <https://keras.io>, 2015.
- [9] Joon Son Chung, Andrew Senior, Oriol Vinyals, and Andrew Zisserman. Lip reading sentences in the wild. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3444–3453, 2017.
- [10] Carlos A. Coello Coello and Carlos E. Mariano Romero. *Evolutionary Algorithms and Multiple Objective Optimization*, pages 277–331. Springer US, Boston, MA, 2002.
- [11] Zbigniew Michalewicz Dipankar Dasgupta. Evolutionary algorithms in engineering applications, 1997.
- [12] D. B. Fogel. An evolutionary approach to the traveling salesman problem, 1988.
- [13] David B. Fogel and Lawrence J. Fogel. An introduction to evolutionary programming. In Jean-Marc Alliot, Evelyne Lutton, Edmund Ronald, Marc Schoenauer, and Dominique Snyers, editors, *Artificial Evolution*, pages 21–33, Berlin, Heidelberg, 1996. Springer Berlin Heidelberg.
- [14] Alex M Fout. *Protein interface prediction using graph convolutional networks*. PhD thesis, Colorado State University, 2017.

- [15] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. *CoRR*, abs/1704.01212, 2017.
- [16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- [17] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Proceedings of the 27th International Conference on Neural Information Processing Systems - Volume 2*, NIPS’14, page 2672–2680, Cambridge, MA, USA, 2014. MIT Press.
- [18] Takuo Hamaguchi, Hidekazu Oiwa, Masashi Shimbo, and Yuji Matsumoto. Knowledge transfer for out-of-knowledge-base entities: A graph neural network approach. *arXiv preprint arXiv:1706.05674*, 2017.
- [19] Geoffrey E. Hinton and R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313:504 – 507, 2006.
- [20] John H. Holland. Genetic algorithms. *Scientific American*, 267(1):66–73, 1992.
- [21] Diederik P. Kingma and Max Welling. An introduction to variational autoencoders. *CoRR*, abs/1906.02691, 2019.
- [22] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *CoRR*, abs/1609.02907, 2016.
- [23] Maciej Komosinski. Evolutionary design of tall structures. 2012.
- [24] Maciej Komosinski. Artificial life and nature-inspired algorithms, 2021. Lecture script.
- [25] Maciej Komosinski and Agnieszka Mensfelt. A flexible dissimilarity measure for active and passive 3d structures and its application in the fitness–distance analysis. In Paul Kaufmann and Pedro A. Castillo, editors, *Applications of Evolutionary Computation*. Springer, Springer, 2019.
- [26] Maciej Komosinski and Adam Rotaru-Varga. Comparison of different genotype encodings for simulated 3d agents. *Artificial Life Journal*, 7(4):395–418, Fall 2001.
- [27] Maciej Komosinski and Szymon Ulatowski. Framsticks genotypes, 2021. http://www.framsticks.com/a/al_genotype.html.
- [28] Maciej Komosinski and Szymon Ulatowski. Framsticks web site, 2021. <http://www.framsticks.com/>.
- [29] Maciej Komosinski and Szymon Ulatowski. Predator prey ecosystem, 2021. <http://www.framsticks.com/wiki/PredatorPreyEcosystem.html>.
- [30] Maciej Komosiński and Adam Rotaru-Varga. Comparison of different genotype encodings for simulated three-dimensional agents. *Artificial life*, 7:395–418, 02 2001.
- [31] John R. Koza and Riccardo Poli. *Genetic Programming*, pages 127–164. Springer US, Boston, MA, 2005.
- [32] Chris Leger. Darwin2k, 2000.
- [33] Paweł Liskowski, Krzysztof Krawiec, Nihat Engin Toklu, and Jerry Swan. Program synthesis as latent continuous optimization: Evolutionary search in neural embeddings. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference*, GECCO ’20, page 359–367, New York, NY, USA, 2020. Association for Computing Machinery.
- [34] D. Marbach, A.J. Ijspeert, and F. Groen. Co-evolution of configuration and control for homogenous modular robots. *Proceedings of the Eighth Conference on Intelligent Autonomous Systems (IAS8)*, 01 2004.

- [35] Hyondong Oh and Yaochu Jin. *Evolving H-GRNs for Morphogenetic Adaptive Pattern Formation of Swarm Robots*, chapter 13, pages 327–361. John Wiley & Sons, Ltd, 2016.
- [36] Adam Santoro, David Raposo, David G. T. Barrett, Mateusz Malinowski, Razvan Pascanu, Peter W. Battaglia, and Timothy P. Lillicrap. A simple neural network module for relational reasoning. *CoRR*, abs/1706.01427, 2017.
- [37] Alessandro Sperduti and Antonina Starita. Supervised neural networks for the classification of structures. *IEEE Transactions on Neural Networks*, 8(3):714–735, 1997.
- [38] Lucas Theis, Wenzhe Shi, Andrew Cunningham, and Ferenc Huszár. Lossy image compression with compressive autoencoders, 2017.
- [39] Vedat Toğan and Ayşe Daloglu. Optimization of 3d trusses with adaptive approach in genetic algorithms. *Engineering Structures*, 28:1019–1027, 06 2006.
- [40] Aäron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew W. Senior, and Koray Kavukcuoglu. Wavenet: A generative model for raw audio. *CoRR*, abs/1609.03499, 2016.
- [41] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks, 2018.
- [42] Zbigniew Wojna, Alexander N. Gorban, Dar-Shyang Lee, Kevin Murphy, Qian Yu, Yeqing Li, and Julian Ibarz. Attention-based extraction of structured information from street view imagery. *CoRR*, abs/1704.03549, 2017.
- [43] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Lukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. Google’s neural machine translation system: Bridging the gap between human and machine translation. *CoRR*, abs/1609.08144, 2016.
- [44] Yongji Wu, Defu Lian, Yiheng Xu, Le Wu, and Enhong Chen. Graph convolutional networks with markov random field reasoning for social spammer detection. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(01):1054–1061, Apr. 2020.
- [45] Xinjie Yu and Mitsuo Gen. *Introduction to Evolutionary Algorithms*. Springer Publishing Company, Incorporated, 2012.
- [46] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. Graph neural networks: A review of methods and applications. *AI Open*, 1:57–81, 2020.



© 2021 Adam Klejda,

Poznań University of Technology
Faculty of Computing and Telecommunication
Institute of Computing Science

Typeset using L^AT_EX